CAN THO UNIVERSITY OF TECHNOLOGY

PHÁT TRIỂN ỨNG DỤNG IOT

(IoT application development)

Instructor : Dinh-Tu Nguyen (Ph.D.) – TS. Nguyễn Đình Tứ

Office: Faculty of Mechanical Engineering

Email: ndtu@ctuet.edu.vn

CHƯƠNG 4: CÁC NỀN TẢNG PHẦN CỨNG VÀ PHẦN MỀM IOT THÔNG DỤNG

4.1: Wireless sensor networks

4.2: Class of wireless sensor networks

4.3: The role of Open source in IoT

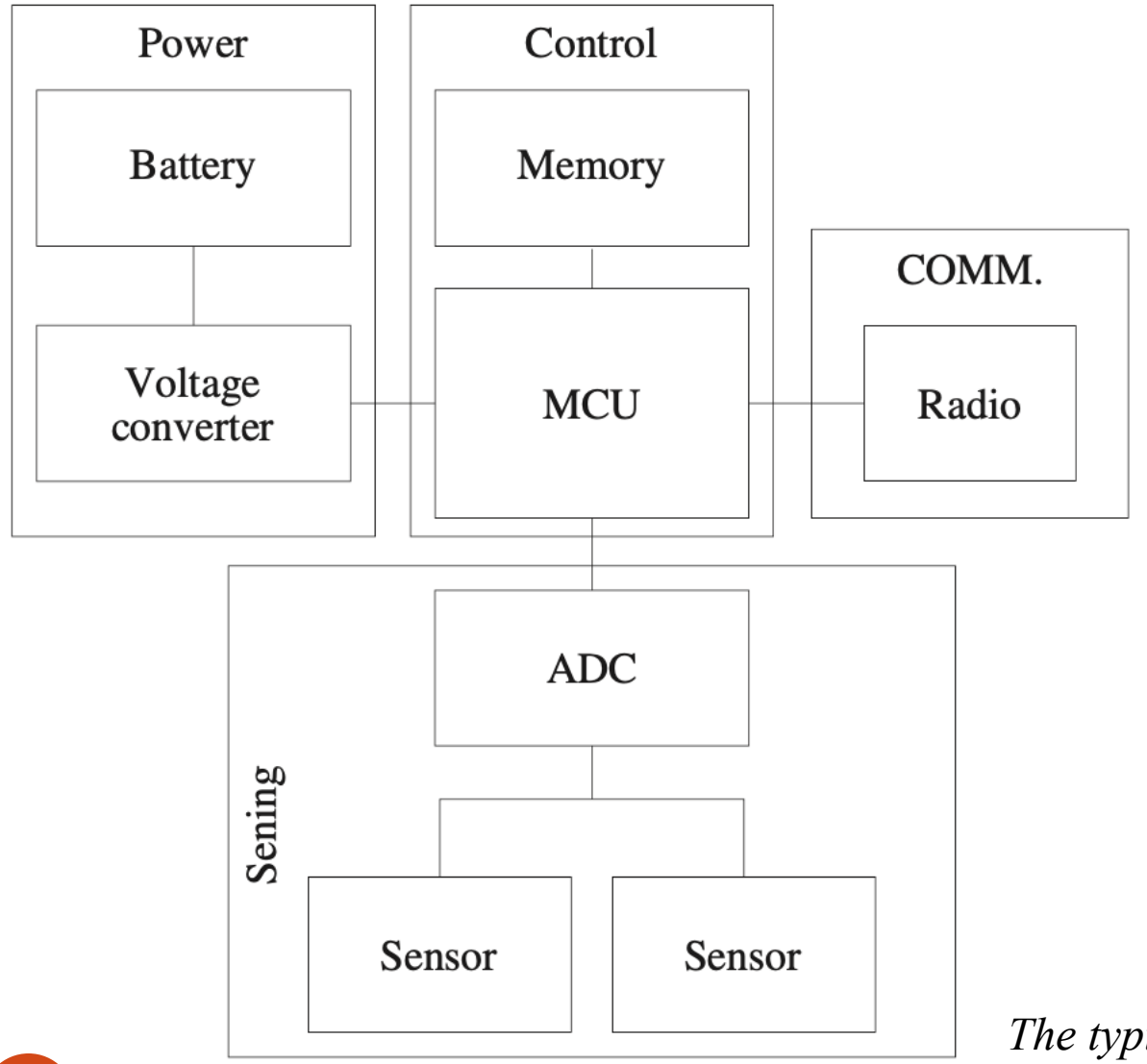
4.4: Các nền tảng phần cứng

4.5: Các nền tảng phần mềm

4.1. Wireless Sensor Networks

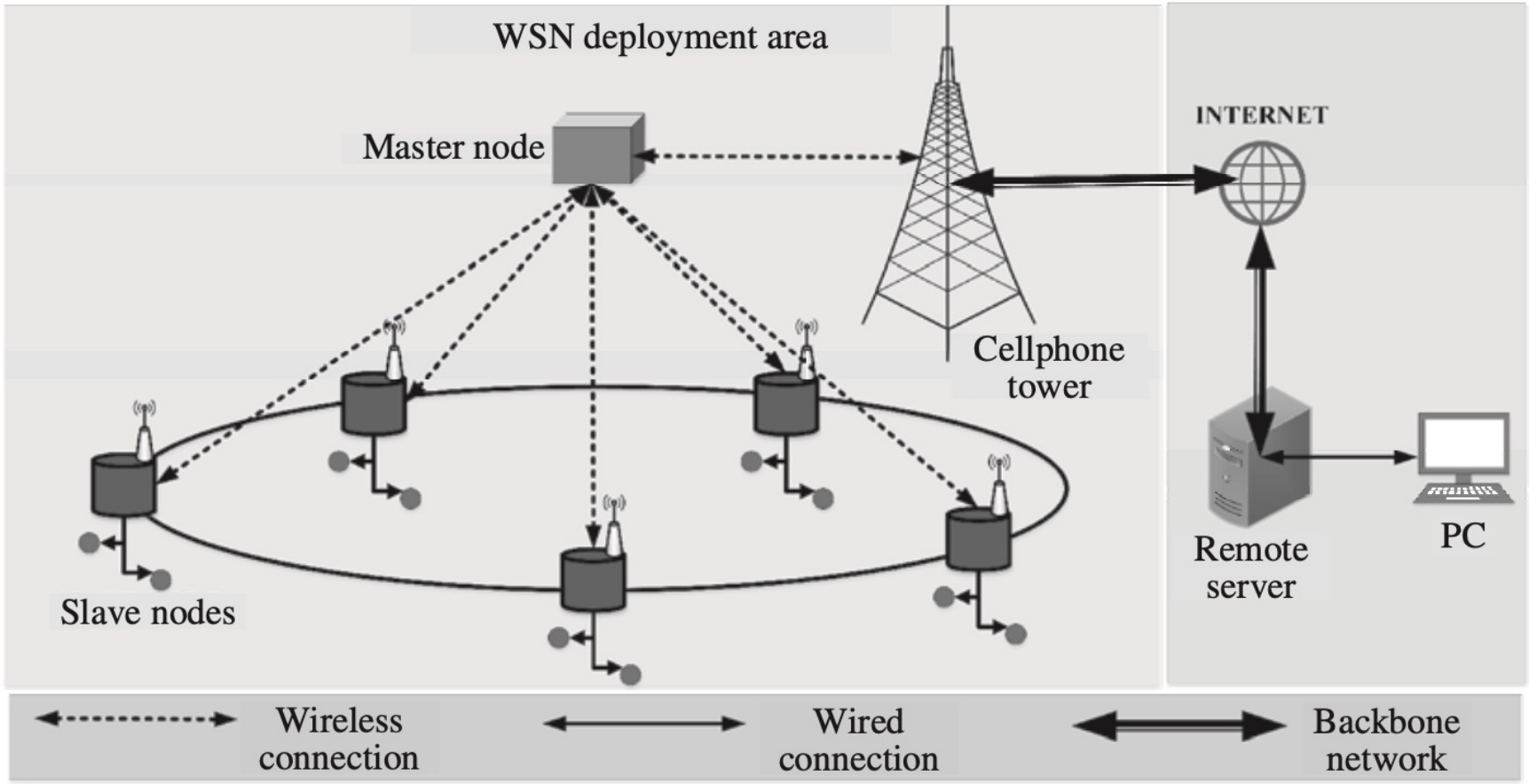
- **Wireless sensor networks** (**WSN**), as the name suggests, is a networking paradigm that makes use of spatially distributed sensors for gathering information concerning the immediate environment of the sensors and collecting the information centrally.
- Here, the sensors are not standalone devices but a combination of sensors, processors, and radio units—referred to as sensor nodes—sensing the environment and communicating the sensed data wirelessly to a remote location, which may or may not be connected to a backbone network.

4.1. Wireless Sensor Networks



The typical constituents of a WSN node

4.1. Wireless Sensor Networks



A typical WSN deployment

4.1. Wireless Sensor Networks

WSNs must have the following distinguishing features

- ✓ **Fault Tolerance**: The occurrence of faults in WSN nodes should not take down the whole WSN implementation, or hamper the transmission of data from non- faulty nodes to the central location.
- ✓ **Scalability**: WSN implementations must have the feature of scalability associated with their architectures and deployments. In the event of a future increase or decrease of sensor node units, the WSN must support the scaling of the infrastructure without changing the whole implementation.

4.1. Wireless Sensor Networks

- ✓ **Long lifetime**: The lifetime or the energy replenishment cycle of WSNs must be long enough to make large-scale applications feasible. WSNs have been used for monitoring remote, harsh, and hard to access environments; in these environments, it is not feasible to regularly replenish the energy source of the WSN nodes, which necessitates the need for long node lifetimes.
- ✓ **Security**: The security of WSNs, if not considered, can easily compromise the security of the whole system, right back to the central server. As WSNs are used for a wide range of applications, some of which are crucial, security is one aspect that must be properly addressed to prevent intrusion and maintain the integrity of the data.

4.1. Wireless Sensor Networks

- ✓ **Programmability**: The programmability of WSNs is important as it ensures the robustness of these systems. WSNs deployed in one application area can be reused for other applications just as easily with a change in sensors and the backend programs associated with it. Programmability also helps in providing a means of adjusting the parameters of the system in the event of a scale-up or scale-down operation.
- ✓ **Affordability**: As WSNs generally require multiple units, typically in the range of tens or hundreds of WSN nodes, the cost of the nodes and its affordability is vastly responsible for the acceptability of the system. Except for some specialized domains such as the military and the industry, where the sensing requirements are quite high and that too in harsh and challenging conditions, the majority of WSN applications are regular. To some extent, the cost of WSN deployments in these regular domains decide the acceptability of a WSN-based solution for that domain.

4.1. Wireless Sensor Networks

- ✓ **Heterogeneity**: The WSNs must support a wide number and various types of sensors and solutions, thus enabling heterogeneity. In the absence of heterogeneity, the WSN will tend to become very application-specific, which in turn would require major customizations even in the event of minor changes to the network or architecture.
- ✓ **Mobility**: WSNs must support the notion of mobility of nodes such that the nodes may be easily relocatable or mobile. Mobility would ensure the rapid deployability of WSN-based solutions in all environments types.

4.2. Class of Wireless Sensor Networks

- ***Military Applications***: WSNs are used for the detection of enemy soldiers, vehicles, intrusion, weapon systems, and armaments.
- ***Health Applications***: WSNs in healthcare are being used to monitor patients in hospitals, ambulances, and homes. Nowadays, a new class of healthcare devices—wearable appliances—enable a user to have a miniature health sensor on them without additional discomfort.
- ***Environmental Applications***: WSNs are used for environmental monitoring of pollution, tracking of wildlife, forests, and others.
- ***Home Applications***: WSNs in the home have given rise to home automation systems and smart home connectivity systems.

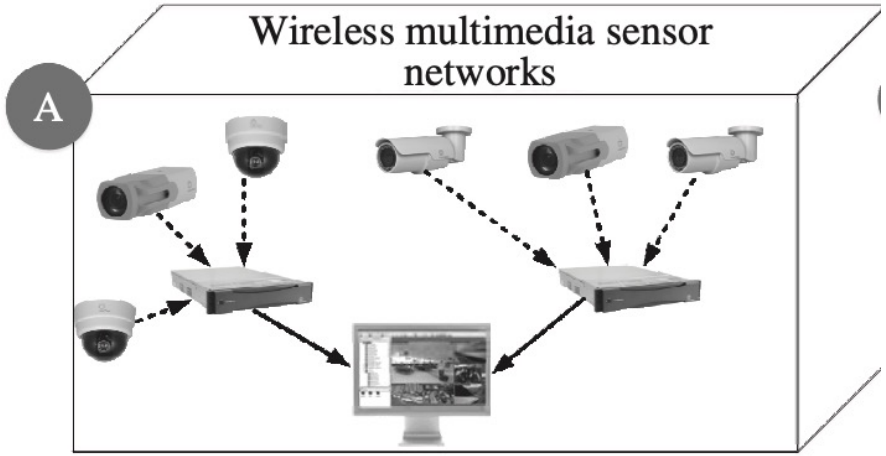
4.2. Class of Wireless Sensor Networks

- ***Commercial Applications***: WSNs are used for tracking of vehicles, packages in transport, logistics, and others.
- ***Industrial Monitoring***: WSNs in industries keep track of various industrial processes, monitor factory floors, ensure worker safety, and perform stock management.

4.2. Class of Wireless Sensor Networks

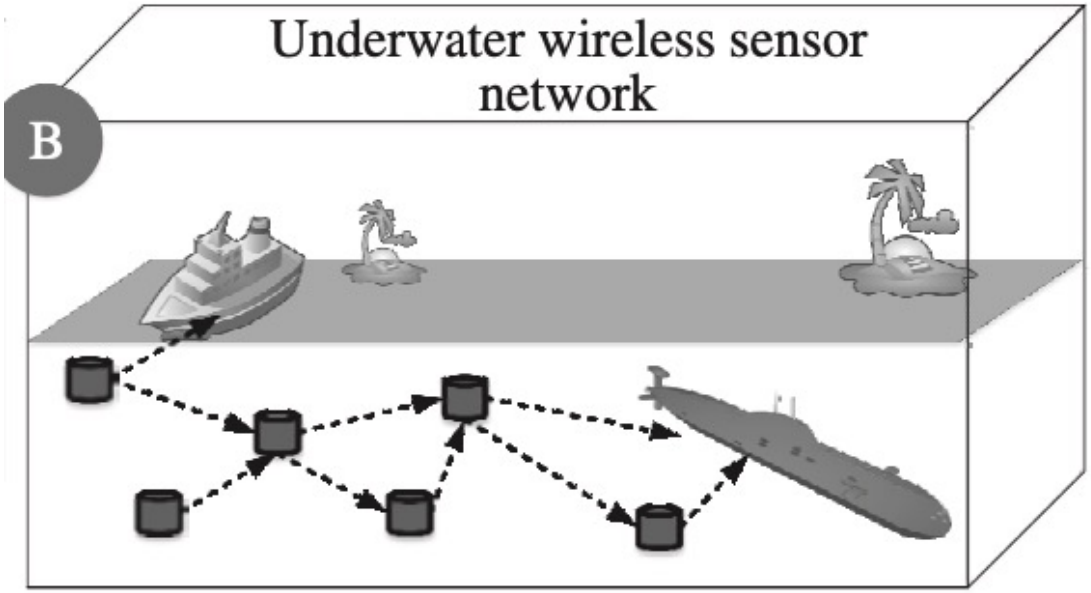
WSNs can be organized broadly into the following domains of implementation:

- **Wireless Multimedia Sensor Networks (WMSN)**: This class of WSNs boasts of the ability to retrieve videos, audios, images, or all three in addition to regular scalar sensor readings. The sensing range and coverage area of a camera- based WMSN are defined by the *field of view* (FOV) of the constituent cameras. Because of their superlative capabilities, WMSNs are popularly sought after in critical domains such as surveillance and road traffic monitoring. However, due to the use of multimedia sensors, the power and processing requirements of this class of WSNs are very high as compared to the other classes of WSNs. Additionally, WMSNs typically demand high network bandwidths for convenient operation.



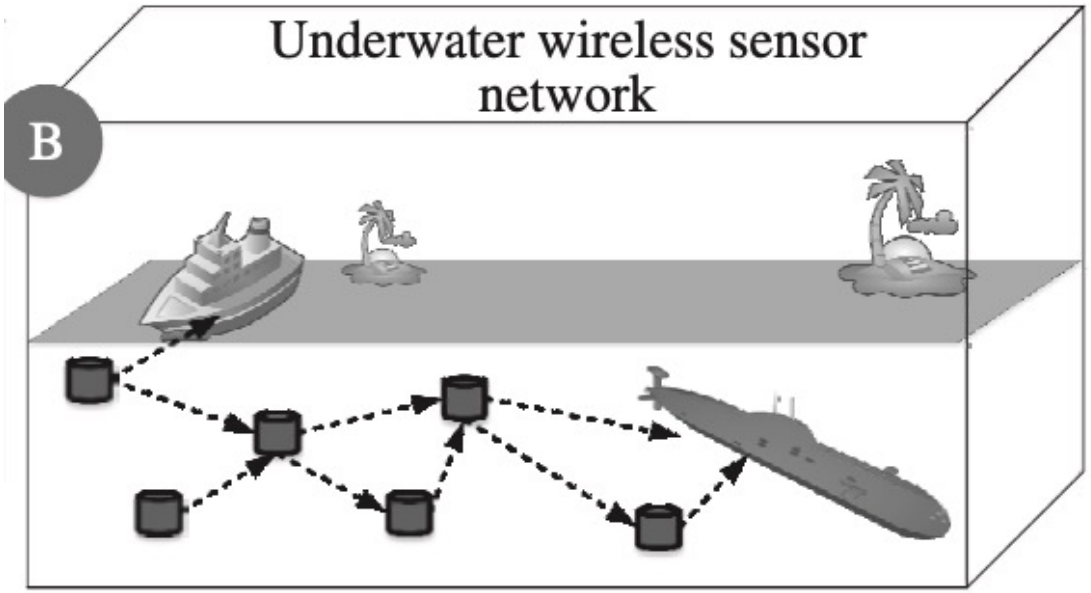
4.2. Class of Wireless Sensor Networks

- **Underwater Sensor Networks (UWSN)**: This class of WSN is designed specifically to work in underwater environments. Compared to terrestrial or aerial environments, wireless underwater communications are severely restricted in terms of data rate, range, and bandwidth due to the high underwater attenuation of electromagnetic signals. The reliable use of light- based underwater communication is also limited in terms of range and noise.



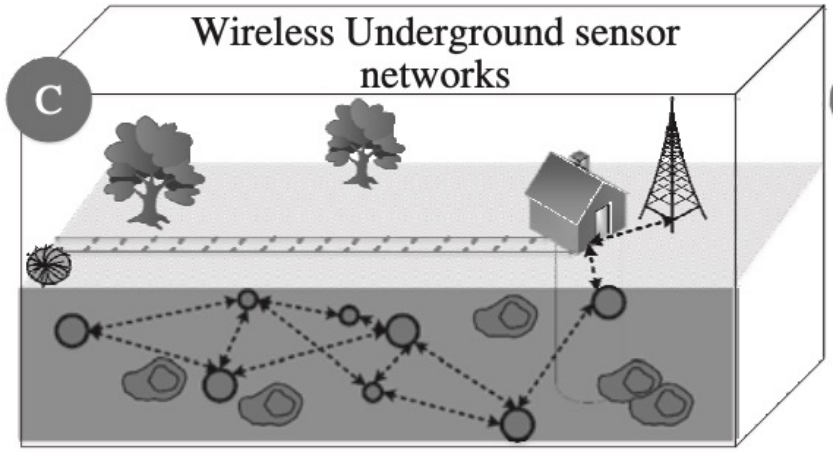
4.2. Class of Wireless Sensor Networks

The most accepted global means of underwater communication is by acoustic waves. However, the long propagation delays and uneven data rate makes it necessary to develop newer topologies and architectures, which can work under the conditions of severe limitations of the physical layer.



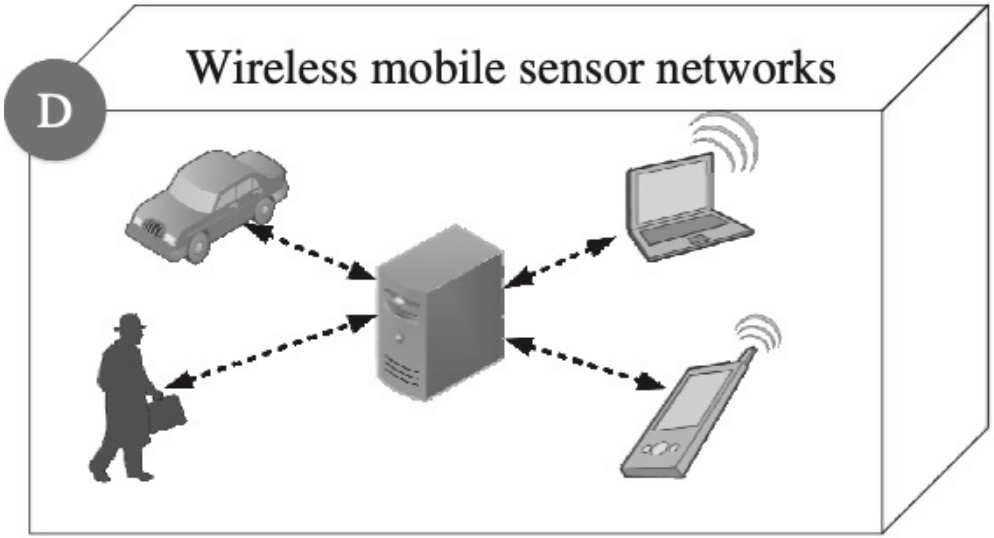
4.2. Class of Wireless Sensor Networks

- **Wireless Underground Sensor Networks (WUSN)**: This class of WSNs is designed to be deployed entirely underground. The underground environment poses challenges of attenuation due to the rocks and minerals in the soil. Another significant problem associated with this class is the need for digging up of the nodes to replenish their energy sources. Typical usage scenarios of this class of WSNs are underground mines and monitoring of underground plumbing systems. WUSNs need denser deployment architectures owing to the limited range of wireless communication in underground environments.



4.2. Class of Wireless Sensor Networks

❖ **Wireless Mobile Sensor Networks (MSN)**: This class of WSNs is characterized by its mobility and low power requirements. The sensor nodes are mobile, which requires them to rapidly connect to networks, disconnect from them, and then again connect to new networks until the nodes are mobile. Typical examples of MSNs include smartphone networks, wearables, vehicular networks, and others.



4.3. The Role of Open Source in IoT

The Open Source Movement

Open source in the computer industry is the publishing of source code or hardware design, with associated licensing that permits the reuse, modification, improvement, and potential commercialization under favorable terms. Example of favorable distribution terms includes the following criteria:

- ***Free Distribution***: Any party may sell or give away the open-source component as part of a larger system without being obligated to pay a royalty or other fee for such sale.
- ***Source Code/Design***: The source code or design must be distributed and made publicly available.
- ***Derived Works***: Derivation and modification of the original open-source component are allowed under the original licensing terms.

4.3. The Role of Open Source in IoT

The Open Source Movement

- While any system can potentially be released under an open source license by its owner, successful open source projects have associated communities of interest that are integral to their success.





4.4. Các nền tảng phần cứng

Thiết bị nhúng Arduino

- Board mạch Arduino đa dạng từ đơn giản tới những sản phẩm phục vụ thiết bị có thể mang theo tới các máy in 3D... Nhiều nhà nghiên cứu, sinh viên thực hiện các dự án với Arduino và cung cấp nhiều hướng dẫn, hỗ trợ online. Một số dòng thiết bị nổi tiếng như Arduino Yun, board Wi-Fi (IEEE 802.11 b/g/n) và Ethernet (IEEE 802.3 10/100Mb/s).



Arduino



Arduino IDE

4.4. Các nền tảng phần cứng

Thiết bị nhúng Arduino

- Board này sử dụng ATmega32, xung clock 16MHz, board này có 1 khe micro-SD, 1 port USB, 7 chan PWM pin, 12 analog input và 1 cổng micro USB.



Arduino



Arduino IDE

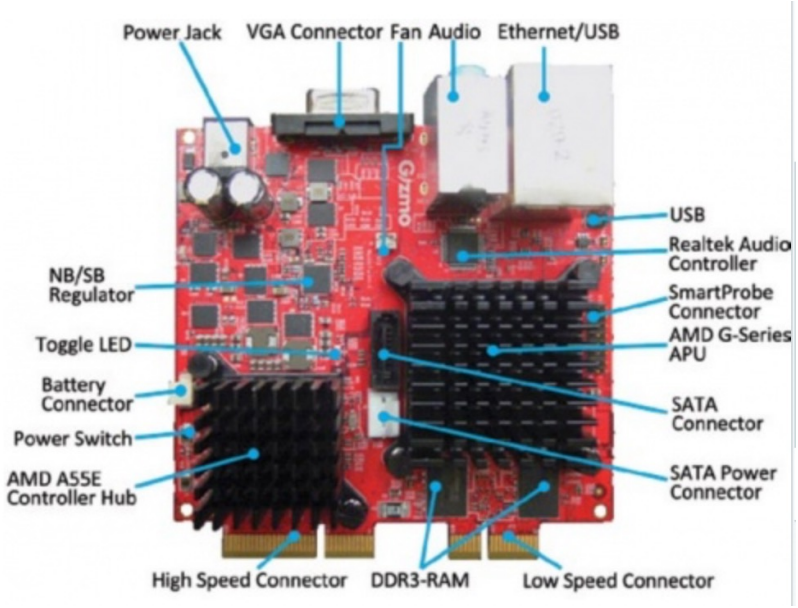
4.4. Các nền tảng phần cứng

IoT Devices- Hardware

- GizmoSphere is an open source development platform for the embedded design community; the site includes code downloads and hardware schematics along with free user guides, specification sheets, and other documentation.



GizmoSphere 2

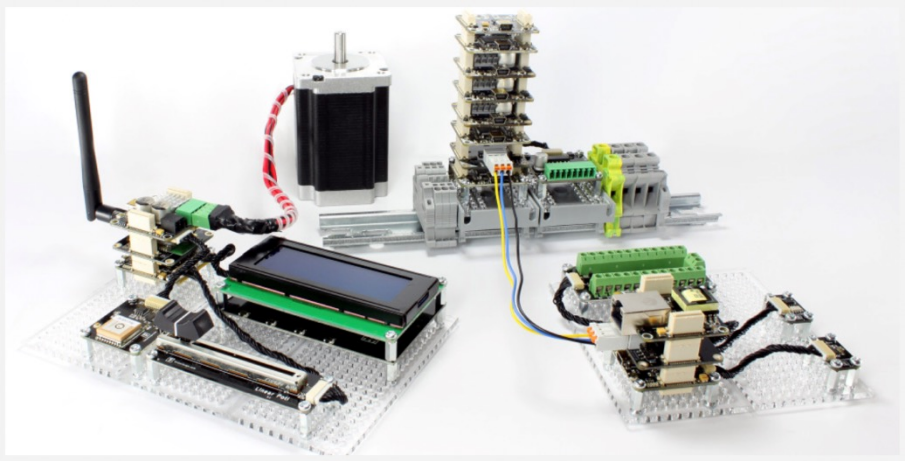


GizmoSphere

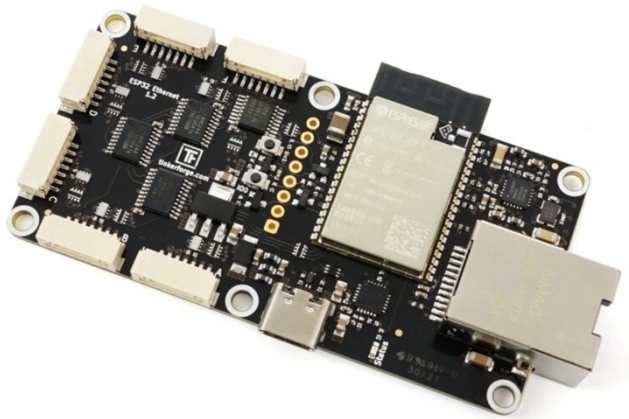
4.4. Các nền tảng phần cứng

IoT Devices- Hardware

- Tinkerforge is a system of open source stackable microcontroller building blocks. It allows the control of motors and reading out sensors with the following programming languages: C, C++, C#, Object Pascal, Java, PHP, Python, and Ruby over a USB or Wi-Fi connection on Windows, Linux, and Mac OS X. All of the hardware is licensed under CERN OHL (CERN Open Hardware License).



Tinkerforge



Tinkerforge stand-alone with ESP32 Bricks

4.4. Các nền tảng phần cứng

IoT Devices- Hardware

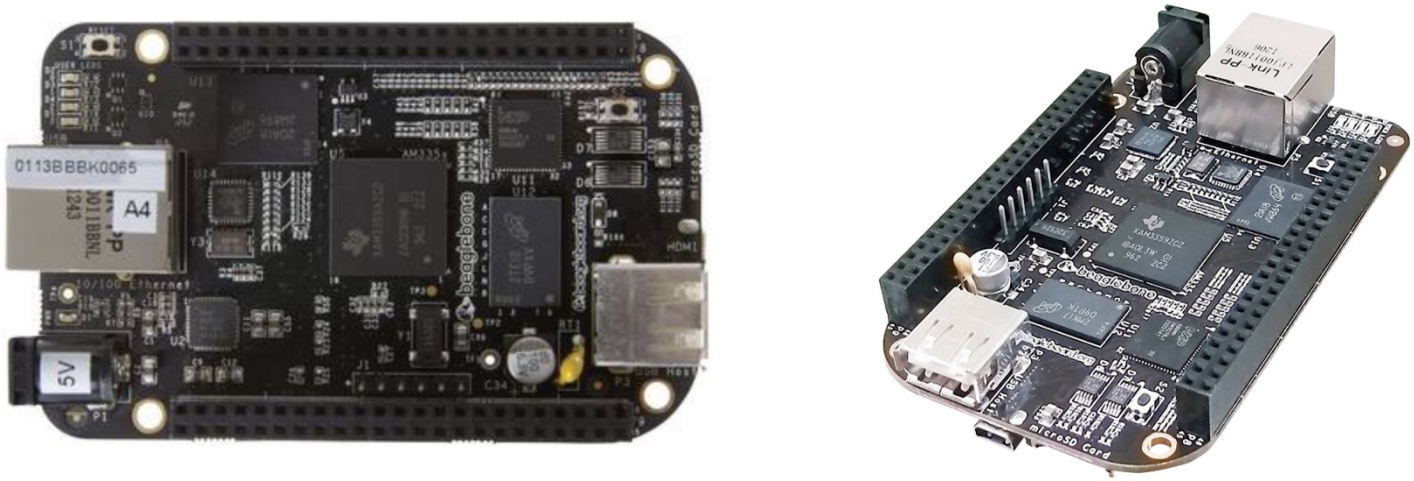


Tinkerforge

4.4. Các nền tảng phần cứng

IoT Devices- Hardware

- BeagleBoard offers credit-card sized computers that can run Android and Linux. Because they have very low-power requirements, they are a good option for IoT devices. Both the hardware designs and the software they run are open source, and BeagleBoard hardware (often sold under the name BeagleBone) is available through a wide variety of distributors.



BeagleBone

4.4. Các nền tảng phần cứng

IoT Devices- Hardware

- As of 2020, the Raspberry Pi is the most used edge device for building IoT solutions among the DIY and maker communities. The affordable price and accessibility made it extremely popular. The computing and storage capabilities of the Raspberry Pi enables people to use it as an edge device within the IoT projects they are building.



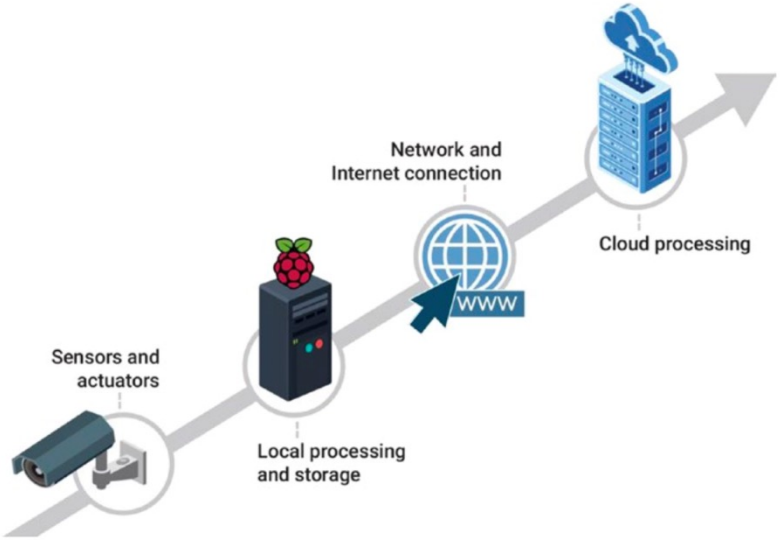
Raspberry Pi B+



Raspberry Pi Zero

4.4. Các nền tảng phần cứng

IoT Devices- Hardware



The Internet of Things stack

- Based on Figure, we notice that the sensors and actuators are placed at the bottom of the stack, as they are the ones communicating directly with the environment. Up the stack, we place the local processing and storage devices, also called edge devices.

- These are computers such as a Raspberry Pi, which are capable of exchanging information with the peripherals and have minimal computing capabilities so they can preprocess the data received from the sensors.

4.4. Các nền tảng phần cứng

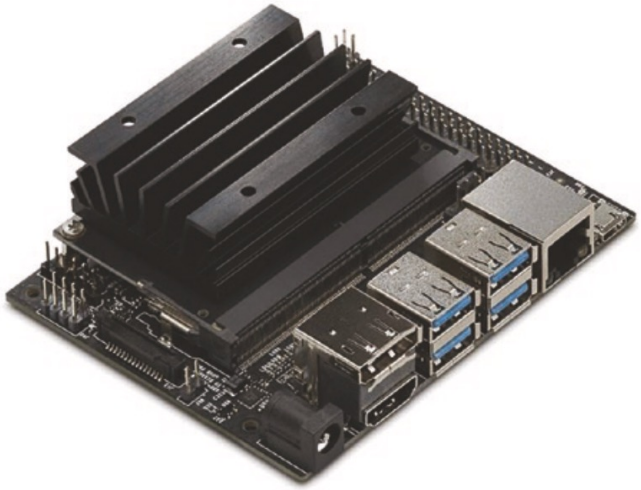
IoT Devices- Hardware



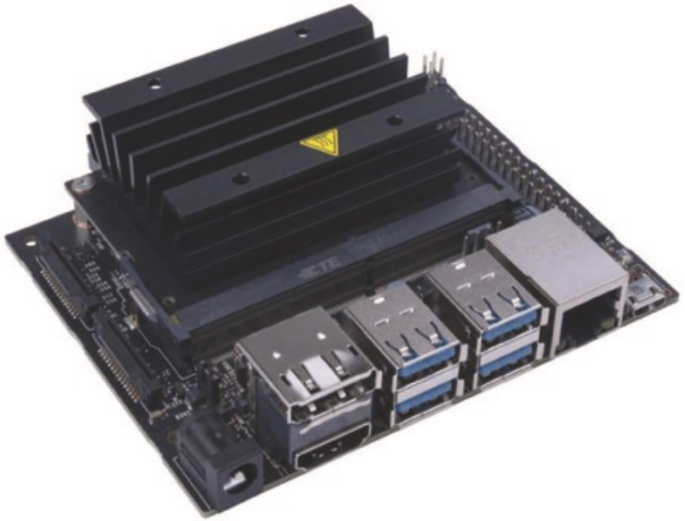
4.4. Các nền tảng phần cứng

IoT Devices- Hardware

- The Nvidia Jetson Nano was announced as a development system in mid-March of 2019. This product is intended for Internet of Things (IoT) makers. The board consists of a CPU with 1.43 GHz and a GPU with 128 cores of the



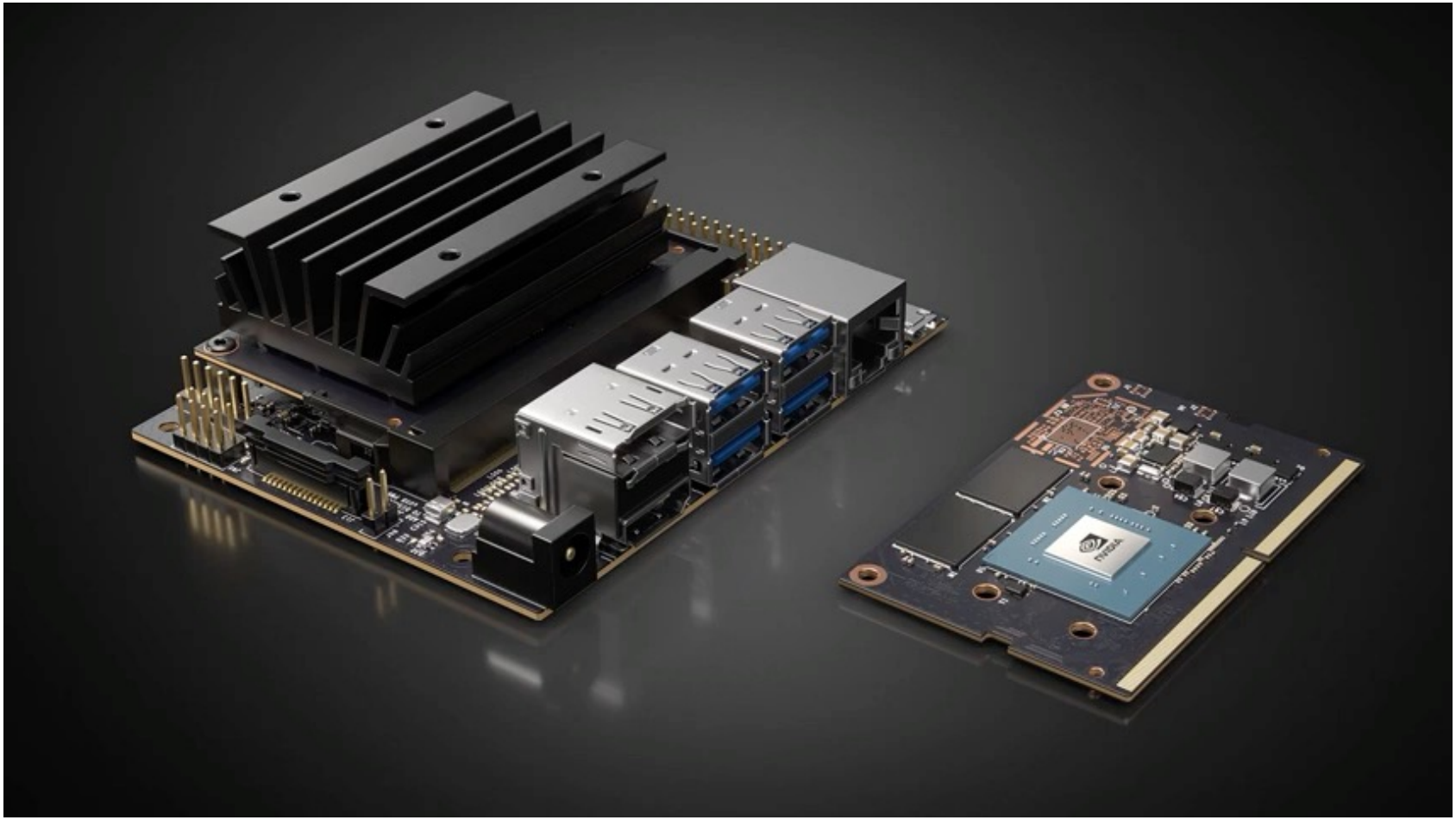
NVDIA Jetson Nano A02



NVDIA Jetson Nano B01

4.4. Các nền tảng phần cứng

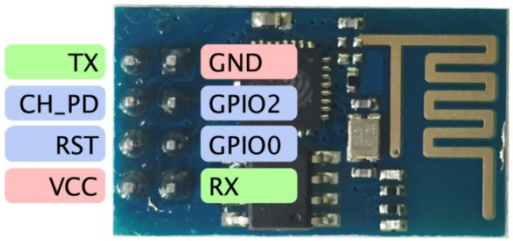
IoT Devices- Hardware



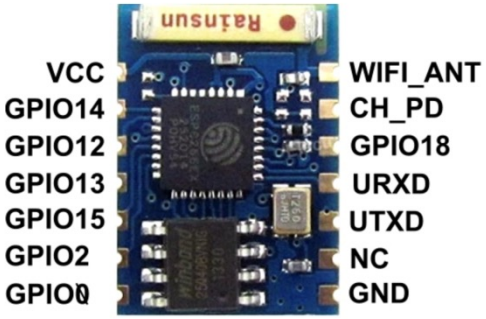
4.4. Các nền tảng phần cứng

IoT Devices- Hardware

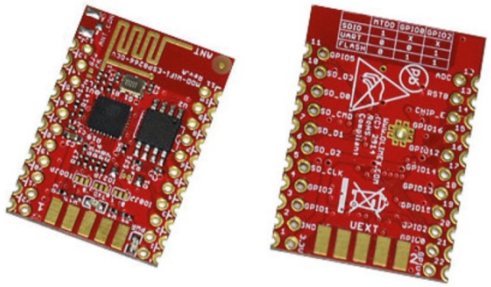
- ESP8266 is a wifi SoC (system on a chip) produced by Espressif Systems . It is an highly integrated chip designed to provide full internet connectivity in a small package. ESP8266 can be used as an external Wifi module, using the standard AT Command set Firmware by connecting it to any microcontroller using the serial UART, or directly serve as a Wifi-enabled microcontroller, by programming a new firmware using the provided SDK.



ESP8266-01



ESP8266-03

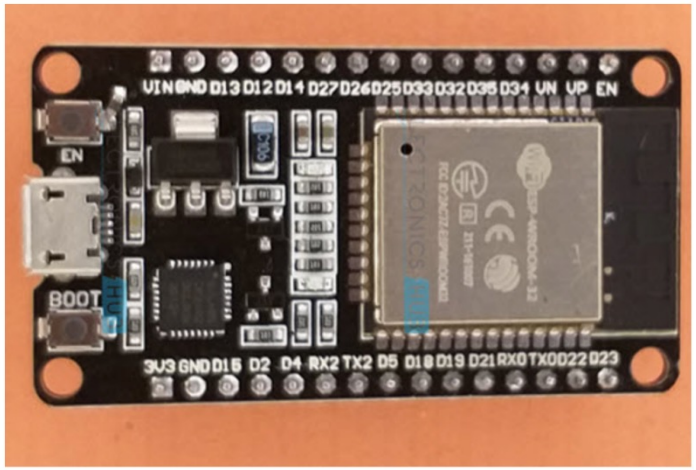


Olimex ESP8266

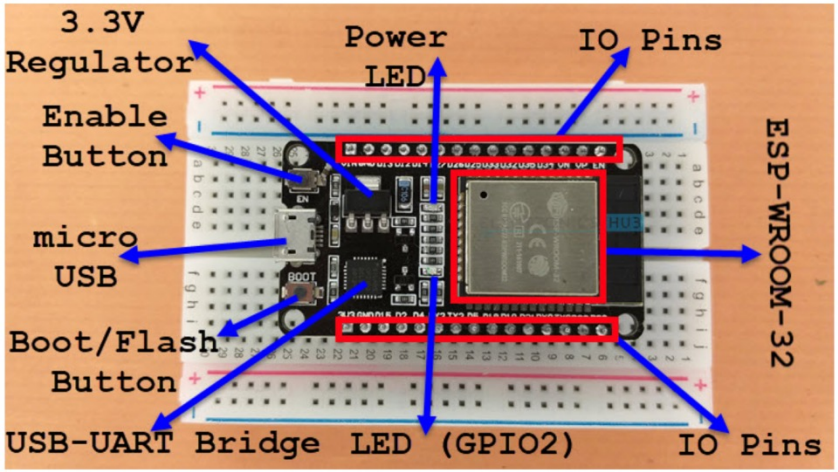
4.4. Các nền tảng phần cứng

IoT Devices- Hardware

- ESP32 is a low-cost System on Chip (SoC) Microcontroller from Espressif Systems, the developers of the famous ESP8266 SoC. It is a successor to ESP8266 SoC and comes in both single-core and dual-core variations of the Tensilica's 32-bit Xtensa LX6 Microprocessor with integrated Wi-Fi and Bluetooth.



ESP32 Devkit board



ESP32 Devkit board with its description

4.4. Các nền tảng phần cứng

IoT Devices- (wearables + IoT)

- The Zerynth IoT Platform is a full set of hardware-software tools designed by IoT experts to enable an easy transition from the prototyping phase to mass production in a flexible, scalable, and efficient way.





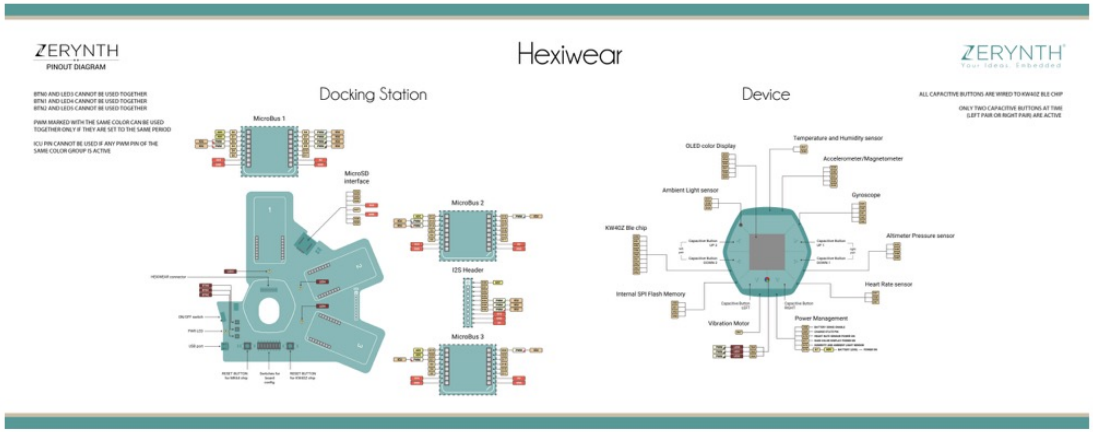
Zerynth IoT platform

4.4. Các nền tảng phần cứng

IoT Devices - (wearables + IoT)

- Zerynth OS is an operating system designed to simplify the development process. It features a Python/C Development environment and seamless integration with other hardware vendors.
- The Zerynth Cloud makes it possible to manage IoT devices, store data, visualize data, and gain insights from a single interface.



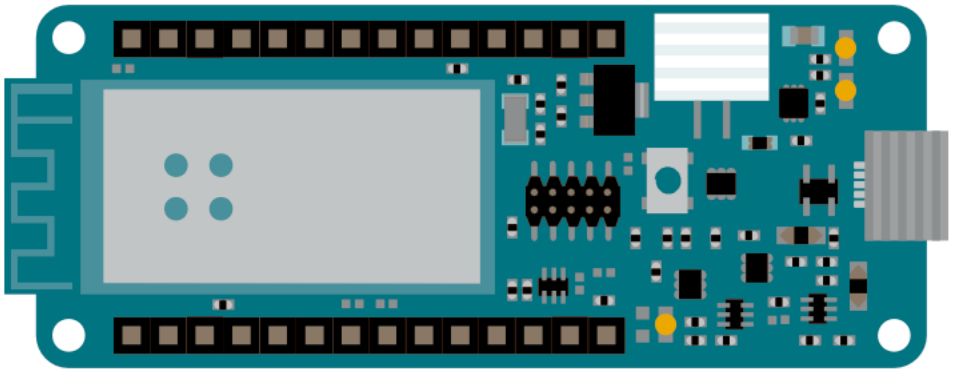


4.4. Các nền tảng phần cứng

IoT Devices - (wearables + IoT)

Alzheimer's is an extremely destructive disease that destroys the very essence of oneself by degrading memory and thinking abilities. Because 1 in 9 people aged 65 and above suffer from it, Abdullah Sadiq decided to build an assistive smartwatch that displays basic reminders, shows the time, monitors the wearer's location, and can even alert caretakers in the event of an emergency. It works by using an Infineon Sensor Hub in conjunction with an Arduino MKR1000 board to read sensor information and send it to either a smartphone or to an IoT platform.



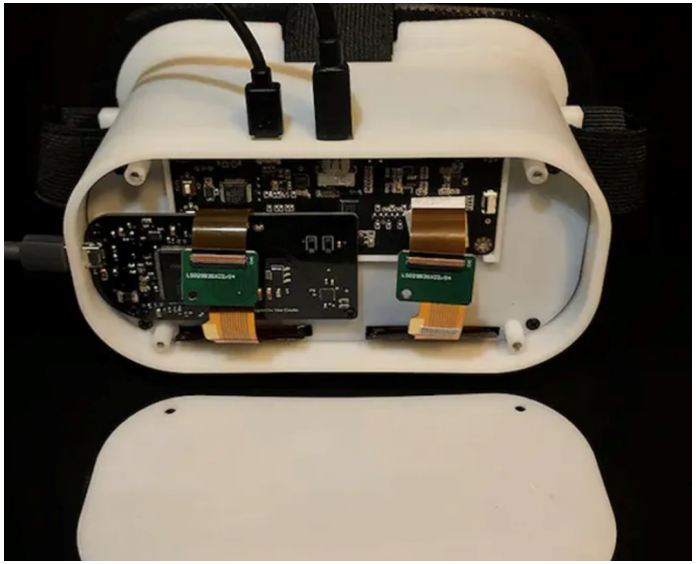


Arduino MKR1000 board

4.4. Các nền tảng phần cứng

IoT Devices - (Open source VR headset)

Probably the most advanced project in this list, the Relativity device is an open source VR headset that supports SteamVR and even body-tracking through the use of a neural network that estimates 3D positions. The headset was made possible thanks to its dual 2K displays that can output images at 120 fps. All of the computational horsepower comes from an Microchip SAM3X8E Cortex-M3 processor mounted to a custom PCB along with an MPU-6050 IMU for motion tracking.



Open source VR headset

4.4. Các nền tảng phần cứng

IoT Devices - (Open source VR headset)

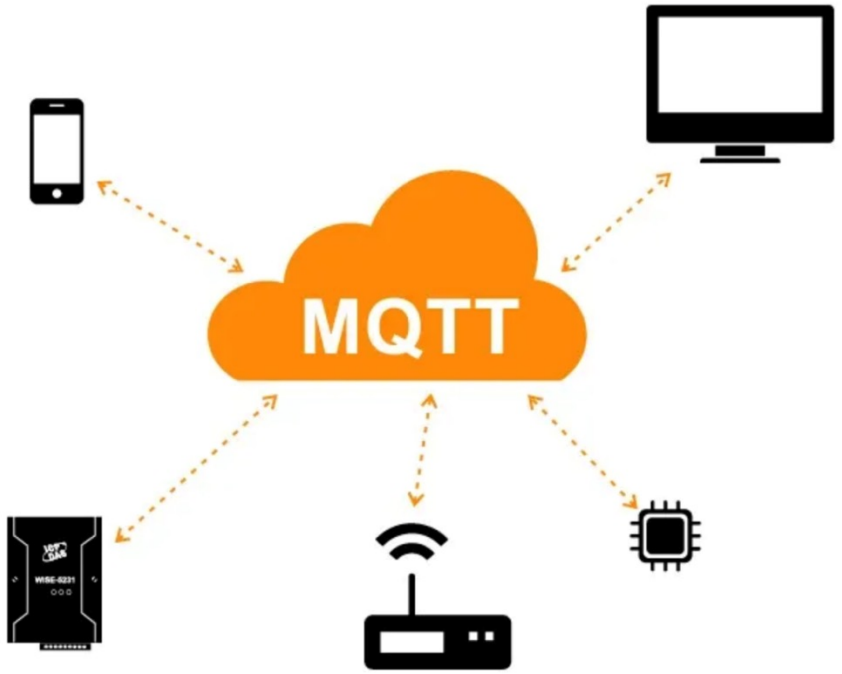


Open source VR headset

4.5. Các nền tảng phần mềm

Giao thức MQTT

- *MQTT* được thiết kế chủ yếu để kết nối các thiết bị bị hạn chết nguồn trên băng thông thấp. Mặc dù đã tồn tại trong hơn một thập kỷ nhưng chỉ khi có sự ra đời của M2M và IoT mới làm cho MQTT trở thành một giao thức phổ biến.



4.5. Các nền tảng phần mềm

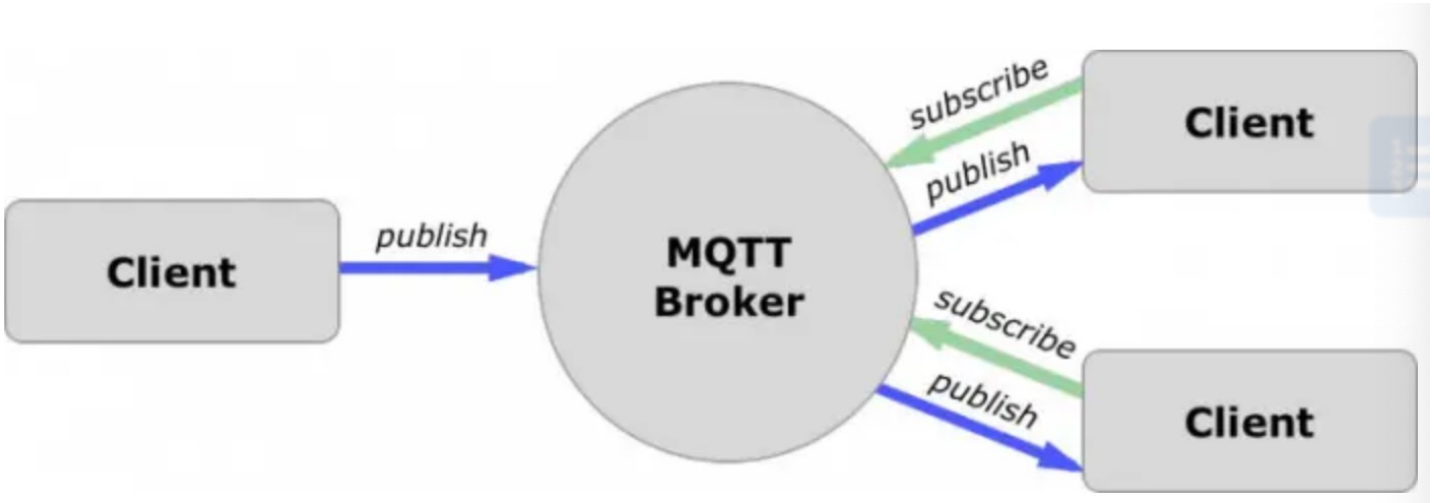
Giao thức MQTT

- *MQTT* (Message Queuing Telemetry Transport) là một giao thức truyền thông điệp (message) theo mô hình publish/subcribe (xuất bản – theo dõi), sử dụng băng thông thấp, độ tin cậy cao và có khả năng hoạt động trong điều kiện đường truyền không ổn định.
- MQTT gồm có các thành phần sau:
  - Client
  - Broker
  - Topic
  - Message
  - Publish
  - Subcribe
  - Connection

4.5. Các nền tảng phần mềm

Giao thức MQTT

- Giống như bất kỳ giao thức internet nào khác, MQTT dựa trên các client và một máy chủ. Tương tự như vậy, máy chủ (broker) là người chịu trách nhiệm xử lý các yêu cầu của client về việc nhận hoặc gửi dữ liệu giữa client và server với nhau.



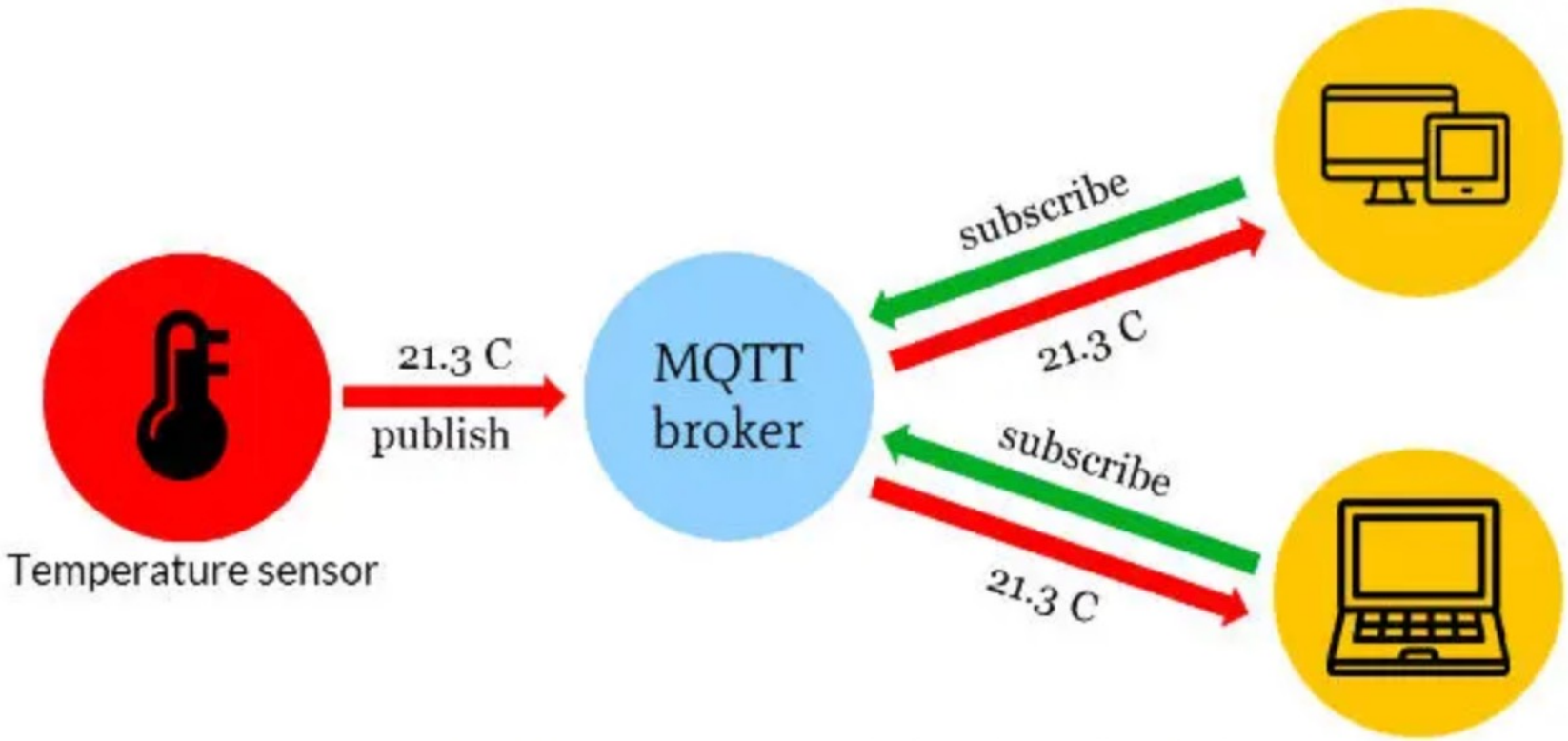
4.5. Các nền tảng phần mềm

Giao thức MQTT

- Giả sử có một thiết bị gắn cảm biến nhiệt độ. Nó sẽ gửi dữ liệu đọc được cho broker. Ở phía bên kia, ứng dụng của điện thoại hoặc PC nhận được giá trị của nhiệt độ này.
- Thiết bị xác định chủ đề (topic) mà nó muốn xuất bản (publish), ví dụ "temp". Sau đó, nó xuất bản thông báo "giá trị nhiệt độ".
- Ứng dụng điện thoại/máy tính PC đăng ký (subcribe) chủ đề "temp". Sau đó, nó nhận được thông báo rằng thiết bị đã xuất bản, đó là giá trị nhiệt độ.

4.5. Các nền tảng phần mềm

Giao thức MQTT



4.5. Các nền tảng phần mềm

Giao thức MQTT

- Có nhiều broker triển khai giao thức MQTT. Một trong những broker phổ biến nhất và thường được sử dụng là ***Mosquitto broker***.



4.5. Các nền tảng phần mềm

Giao thức MQTT

- Mosquitto là một broker thông điệp nguồn mở thực hiện giao thức MQTT. Nó gọn nhẹ và phù hợp để sử dụng trên tất cả các thiết bị từ một bo mạch đơn giản như Arduino, ESP8266 và ESP32 đến máy tính và máy chủ đầy đủ.
- Nhưng thay vì sử dụng Mosquitto trên máy tính cục bộ, chúng ta sẽ cần phải sử dụng một máy chủ dựa trên đám mây để triển khai broker Mosquitto. Điều đó là cần thiết để làm cho các dự án IoT có thể kiểm soát được trên Internet.
- Có rất nhiều broker mosquitto dựa trên đám mây như: ThingMQ, ThingStudio, MQTT.io, Heroku, CloudMQTT.

4.5. Các nền tảng phần mềm

Blynk

- *Blynk* là một nền tảng với các ứng dụng iOS và Android để điều khiển Arduino, Raspberry Pi và các ứng dụng tương tự qua Internet.
- Blynk là một bảng điều khiển kỹ thuật số nhờ đó bạn có thể xây dựng giao diện đồ họa cho dự án của mình bằng cách kéo và thả các widget.



- Blynk không bị ràng buộc với một số board mạch hoặc shield cụ thể. Thay vào đó, nó hỗ trợ phần cứng mà bạn lựa chọn. Cho dù Arduino hoặc Raspberry Pi của bạn được liên kết với Internet qua Wi-Fi. Blynk sẽ giúp bạn online và sẵn sàng cho IoT.

4.5. Các nền tảng phần mềm

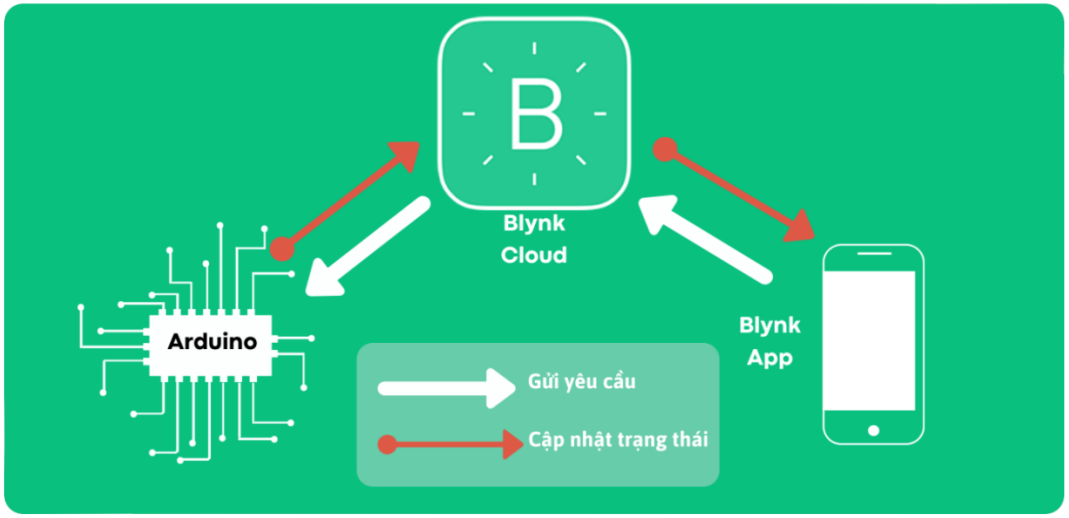
Blynk có thể làm được những gì?

- Triển khai một hệ thống nhà thông minh, điều khiển giám sát từ xa, v.v... Giới hạn duy nhất là khả năng cũng như sự sáng tạo của bạn.
- Các Widget như: nút ấn, hiện thị giá trị, thanh kéo giá trị, vẽ sơ đồ, v.v... Cho phép điều khiển GPIO của thiết bị bo mạch, từ đó bật tắt relay, điều khiển các thiết bị trong nhà.
- Điểm mạnh của Blynk là có thể kết nối ở bất kỳnơi nào có mạng.
- Có thể chia sẻ bảng điều khiển cho nhiều người dùng để sử dụng chung 1 hệ thống.

4.5. Các nền tảng phần mềm

Blynk có thể làm được những gì?

- Thiết bị arduino chịu trách nhiệm điều khiển các thiết bị trong nhà thông qua relay lắp trực tiếp vào các GPIO của arduino, hoặc bất kì kiểu giao tiếp nào khác nếu bạn đủ khả năng (RF, Uart). Thiết bị này yêu cầu phải kết nối mạng (ESP8266, ESP32, …)
- App Blynk từ điện thoại thông qua đám mây sẽ gửi tín hiệu đến board và ngược lại, các board arduino được đánh dấu bằng 1 token - chữ ký số, tạo ra trong quá trình setup ứng dụng.



4.5. Các nền tảng phần mềm

B



Arduino Programming Language

A Tour of Open-Source Activities in IoT

What is Physical Computing?

- Physical Computing uses electronics to prototype new materials for designers and artists.
- It involves the design of interactive objects that can communicate with humans using sensors and actuators controlled by a behaviour implemented as software running inside a microcontroller (a small computer on a single chip).

A Tour of Open-Source Activities in IoT

What is Physical Computing?

- In the past, using electronics meant having to deal with engineers all the time, and building circuits one small component at the time; these issues kept creative people from playing around with the medium directly. Most of the tools were meant for engineers and required extensive knowledge. In recent years, microcontrollers have become cheaper and easier to use, allowing the creation of better tools.
- The progress that we have made with Arduino is to bring these tools one step closer to the novice, allowing people to start building stuff after only two or three days of a workshop.
- With Arduino, a designer or artist can get to know the basics of electronics and sensors very quickly and can start building prototypes with very little investment.

A Tour of Open-Source Activities in IoT

The Arduino Way

- The Arduino philosophy is based on making designs rather than talking about them. It is a constant search for faster and more powerful ways to build better prototypes. We have explored many prototyping techniques and developed ways of thinking with our hands.
- Classic engineering relies on a strict process for getting from A to B; the Arduino Way delights in the possibility of getting lost on the way and finding C instead.
- This is the tinkering process that we are so fond of—playing with the medium in an open-ended way and finding the unexpected. In this search for ways to build better prototypes, we also selected a number of software packages that enable the process of constant manipulation of the software and hardware medium.

A Tour of Open-Source Activities in IoT

Prototyping

- Prototyping is at the heart of the Arduino Way: we make things and build objects that interact with other objects, people, and networks. We strive to find a simpler and faster way to prototype in the cheapest possible way.
- A lot of beginners approaching electronics for the first time think that they have to learn how to build everything from scratch. This is a waste of energy: what you want is to be able to confirm that something's working very quickly so that you can motivate yourself to take the next step or maybe even motivate somebody else to give you a lot of cash to do it.
- This is why we developed "opportunistic prototyping": why spend time and energy building from scratch, a process that requires time and in-depth technical knowledge, when we can take ready-made devices and hack them in order to exploit the hard work done by large companies and good engineers?

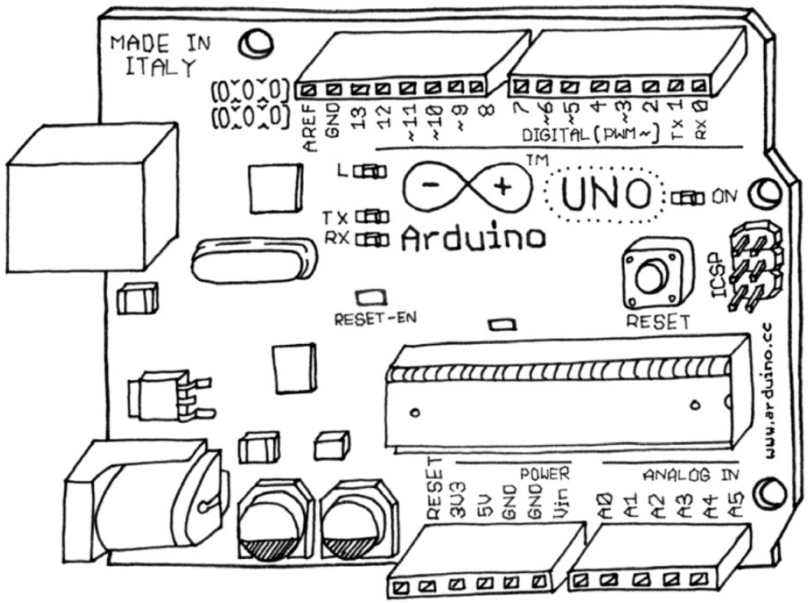
A Tour of Open-Source Activities in IoT

The Arduino Platform

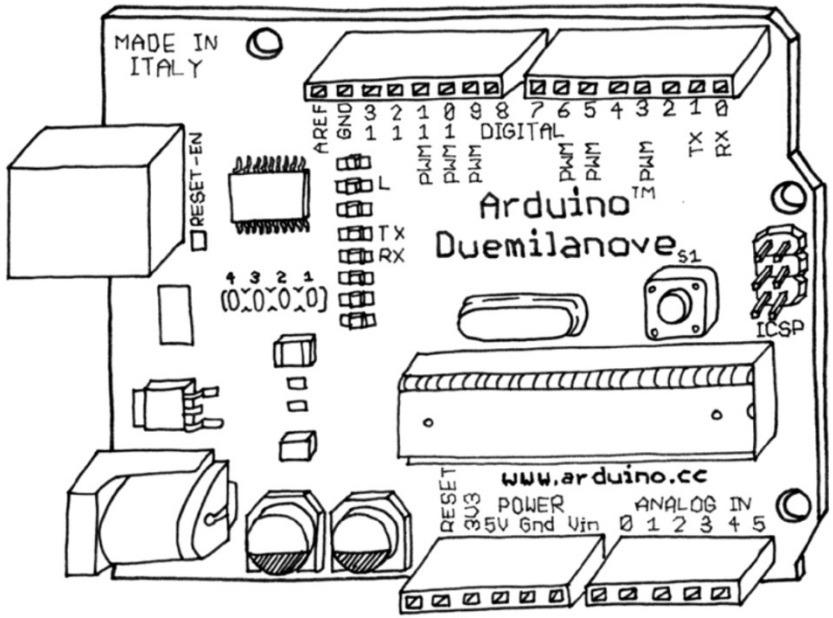
- Arduino is composed of two major parts: the Arduino board, which is the piece of hardware you work on when you build your objects; and the Arduino IDE, the piece of software you run on your computer.
- **14 Digital IO pins (pins 0–13)** These can be inputs or outputs, which is specified by the sketch you create in the IDE.
- **6 Analogue In pins (pins 0–5)** These dedicated analogue input pins take analogue values (i.e., voltage readings from a sensor) and convert them into a number between 0 and 1023.
- **6 Analogue Out pins (pins 3, 5, 6, 9, 10, and 11)** These are actually six of the digital pins that can be reprogrammed for analogue output using the sketch you create in the IDE.

A Tour of Open-Source Activities in IoT

The Arduino Platform



The Arduino Uno



The Arduino Duemilanove

A Tour of Open-Source Activities in IoT

The Software (IDE)

- The IDE (Integrated Development Environment) is a special program running on your computer that allows you to write sketches for the Arduino board in a simple language modeled after the Processing (www.processing.org) language.

The programming cycle on Arduino is basically as follows:

- Plug your board into a USB port on your computer.
- Write a sketch that will bring the board to life.
- Upload this sketch to the board through the USB connection and wait a couple of seconds for the board to restart.
- The board executes the sketch that you wrote.

A Tour of Open-Source Activities in IoT

Installing Arduino on Your Computer

- To program the Arduino board, you must first download the development environment (the IDE) from here: www.arduino.cc/en/Main/Software. Choose the right version for your operating system.



Arduino



Arduino IDE

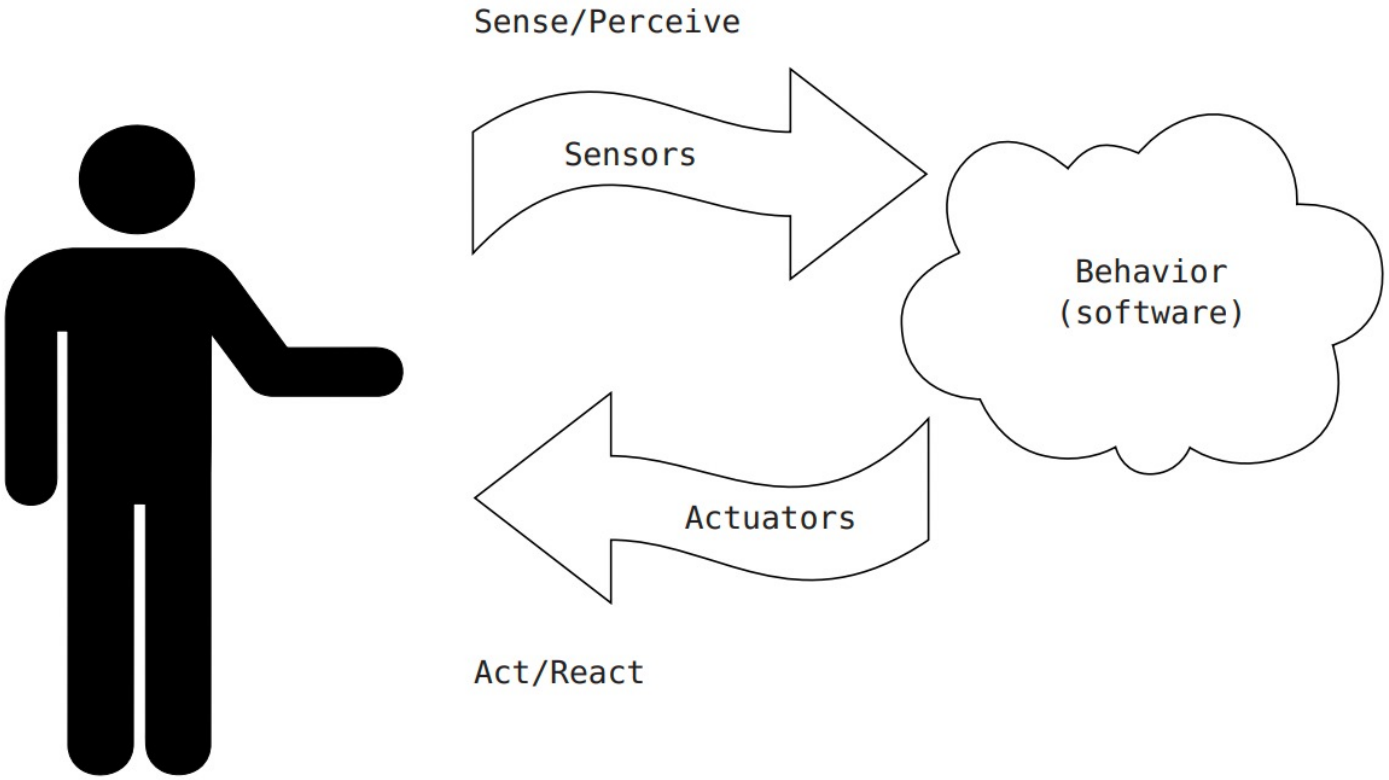
A Tour of Open-Source Activities in IoT

Anatomy of an Interactive Device

- All of the objects we will build using Arduino follow a very simple pattern that we call the "Interactive Device". The Interactive Device is an electronic circuit that is able to sense the environment using sensors (electronic components that convert real-world measurements into electrical signals).
- The device processes the information it gets from the sensors with behaviour that's implemented as software. The device will then be able to interact with the world using actuators, electronic components that can convert an electric signal into a physical action.

A Tour of Open-Source Activities in IoT

Anatomy of an Interactive Device



The interactive device

A Tour of Open-Source Activities in IoT

Sensors and Actuators

- Sensors and actuators are electronic components that allow a piece of electronics to interact with the world.
- As the microcontroller is a very simple computer, it can process only electric signals (a bit like the electric pulses that are sent between neurons in our brains). For it to sense light, temperature, or other physical quantities, it needs something that can convert them into electricity. In our body, for example, the eye converts light into signals that get sent to the brain using nerves. In electronics, we can use a simple device called a light-dependent resistor (an LDR or photoresistor) that can measure the amount of light that hits it and report it as a signal that can be understood by the microcontroller.

A Tour of Open-Source Activities in IoT

STRUCTURE

An Arduino sketch runs in two parts:

`void setup()`

This is where you place the initialisation code—the instructions that set up the board before the main loop of the sketch starts.

`void loop()`

This contains the main code of your sketch. It contains a set of instructions that get repeated over and over until the board is switched off.

A Tour of Open-Source Activities in IoT

SPECIAL SYMBOLS

Arduino includes a number of symbols to delineate lines of code, comments, and blocks of code.

; (semicolon)

Every instruction (line of code) is terminated by a semicolon. This syntax lets you format the code freely. You could even put two instructions on the same line, as long as you separate them with a semicolon. (However, this would make the code harder to read.)

Example:

```
delay(100);
```

A Tour of Open-Source Activities in IoT

{} (curly braces)

This is used to mark blocks of code. For example, when you write code for the *loop()* function, you have to use curly braces before and after the code.

Example:

```
void loop() {
   Serial.println("ciao");
}
```

comments

These are portions of text ignored by the Arduino processor, but are extremely useful to remind yourself (or others) of what a piece of code does.

There are two styles of comments in Arduino:

```
// single-line: this text is ignored until the end of the line
/* multiple-line:
   you can write
   a whole poem in here
*/
```

A Tour of Open-Source Activities in IoT

CONSTANTS

Arduino includes a set of predefined keywords with special values. **HIGH** and **LOW** are used, for example, when you want to turn on or off an Arduino pin. **INPUT** and **OUTPUT** are used to set a specific pin to be either and input or an output

true and **false** indicate exactly what their names suggest: the truth or falsehood of a condition or expression.

A Tour of Open-Source Activities in IoT

VARIABLES

Variables are named areas of the Arduino's memory where you can store data that you can use and manipulate in your sketch. As the name suggests, they can be changed as many times as you like.

Because Arduino is a very simple processor, when you declare a variable you have to specify its type. This means telling the processor the size of the value you want to store.

Here are the *datatypes* that are available:

boolean

Can have one of two values: true or false.

char

Holds a single character, such as A. Like any computer, Arduino stores it as a number, even though you see text. When chars are used to store numbers, they can hold values from –128 to 127.

A Tour of Open-Source Activities in IoT

byte
Holds a number between 0 and 255. As with chars, bytes use only one byte of memory.

int
Uses 2 bytes of memory to represent a number between –32,768 and 32,767; it's the most common data type used in Arduino.

unsigned int
Like int, uses 2 bytes but the **unsigned** prefix means that it can't store negative numbers, so its range goes from 0 to 65,535.

long
This is twice the size of an **int** and holds numbers from –2,147,483,648 to 2,147,483,647.

A Tour of Open-Source Activities in IoT

unsigned long
Unsigned version of **long**; it goes from 0 to 4,294,967,295.

float
This quite big and can hold floating-point values, a fancy way of saying that you can use it to store numbers with a decimal point in it. It will eat up 4 bytes of your precious RAM and the functions that can handle them use up a lot of code memory as well. So use **float**s sparingly.

double
Double-precision floating-point number, with a maximum value of $1.7976931348623157 \times 10^{308}$. Wow, that's huge!

string
A set of ASCII characters that are used to store textual information (you might use a string to send a message via a serial port, or to display on

an LCD display). For storage, they use one byte for each character in the string, plus a null character to tell Arduino that it's the end of the string. The following are equivalent:

```
char string1[]  = "Arduino"; // 7 chars + 1 null char
char string2[8] = "Arduino"; // Same as above
```

array

A list of variables that can be accessed via an index. They are used to build tables of values that can easily be accessed. For example, if you want to store different levels of brightness to be used when fading an LED, you could create six variables called light01, light02, and so on. Better yet, you could use a simple array like:

```
int light[6] = {0, 20, 50, 75, 100};
```

The word "array" is not actually used in the variable declaration: the symbols [] and {} do the job.

CONTROL STRUCTURES

Arduino includes keywords for controlling the logical flow of your sketch.

if . . . else

This structure makes decisions in your program. *if* must be followed by a question specified as an expression contained in parentheses. If the expression is true, whatever follows will be executed. If it's false, the block of code following else will be executed. It's possible to use just *if* without providing an *else* clause.

Example:

```
if (val == 1) {
  digitalWrite(LED,HIGH);
}
```

A Tour of Open-Source Activities in IoT

for

Lets you repeat a block of code a specified number of times.

Example:

```
for (int i = 0; i < 10; i++) {
    Serial.print("ciao");
}
```

switch case

The *if* statement is like a fork in the road for your program. *switch case* is like a massive roundabout. It lets your program take a variety of directions depending on the value of a variable. It's quite useful to keep your code tidy as it replaces long lists of *if* statements.

Example:

```
switch (sensorValue) {
    case 23:
      digitalWrite(13,HIGH);
      break;
    case 46:
      digitalWrite(12,HIGH);
      break;
    default: // if nothing matches this is executed
      digitalWrite(12,LOW);
      digitalWrite(13,LOW);
}
```

A Tour of Open-Source Activities in IoT

while

Similar to *if*, this executes a block of code while a certain condition is true.

Example:

```
// blink LED while sensor is below 512
sensorValue = analogRead(1);
while (sensorValue < 512) {
   digitalWrite(13,HIGH);
   delay(100);
   digitalWrite(13,HIGH);
   delay(100);
   sensorValue = analogRead(1);
}
```

A Tour of Open-Source Activities in IoT

ARITHMETIC AND FORMULAS

You can use Arduino to make complex calculations using a special syntax. + and – work like you've learned in school, and multiplication is represented with an * and division with a /.

There is an additional operator called "modulo" (%), which returns the remainder of an integer division. You can use as many levels of parentheses as necessary to group expressions. Contrary to what you might have learned in school, square brackets and curly brackets are reserved for other purposes (array indexes and blocks, respectively).

Examples:

```
a =  2 + 2;
light = ((12 * sensorValue) - 5 ) / 2;
remainder = 3 % 2; // returns 1
```

COMPARISON OPERATORS

When you specify conditions or tests for *if*, *while*, and *for* statements, these are the operators you can use:

==	equal to
!=	not equal to
<	less than
>	greater than
<=	less than or equal to
>=	greater than or equal to

BOOLEAN OPERATORS

These are used when you want to combine multiple conditions. For example, if you want to check whether the value coming from a sensor is between 5 and 10, you would write:

```
if ((sensor => 5) && (sensor <=10))
```

There are three operators: and, represented with **&&**; or, represented with **||**; and finally not, represented with **!**.

COMPOUND OPERATORS

These are special operators used to make code more concise for some very common operations like incrementing a value.

For example, to increment *value* by 1 you would write:

```
value = value +1;
```

but using a compound operator, this becomes:

```
value++;
```

increment and decrement (–– and ++)

These increment or decrement a value by 1. Be careful: if you write *i*++ this increments *i* by 1 and evaluates to the equivalent of *i+1*; ++*i* evaluates to the value of *i* and **then** increments *i*. The same applies to ––.

+= , –=, *= and /=

These make it shorter to write certain expressions. The following two expressions are equivalent:

```
a = a + 5;
a += 5;
```

A Tour of Open-Source Activities in IoT

MATH FUNCTIONS

Arduino includes many common mathematical and trigonometric functions:

min(x, y)

Returns the smaller of *x* and *y*.

Example:

```
val = min(10,20); // val is now 10
```

max(x, y)

Returns the larger of *x* and *y*.

Example:

```
val = max(10,20); // val is now 20
```

abs(x)

Returns the absolute value of *x*, which turns negative numbers into positive. If *x* is 5 it will return 5, but if *x* is –5, it will still return 5.

Example:

```
val = abs(-5); // val is now 5
```

constrain(x, a, b)

Returns the value of *x*, constrained between a and b. If *x* is less than *a*, it will just return *a* and if *x* is greater than *b*, it will just return *b*.

Example:

```
val = constrain(analogRead(0), 0, 255); // reject values bigger than 255
```

map(value, fromLow, fromHigh, toLow, toHigh)

Maps a value in the range *fromLow* and *maxLow* to the range *toLow* and *toHigh*. Very useful to process values from analogue sensors.

Example:

```
val = map(analogRead(0),0,1023,100, 200); // maps the value of
                                          // analog 0 to a value
                                          // between 100 and 200
```

A Tour of Open-Source Activities in IoT

double pow(base, exponent)
Returns the result of raising a number (*base*) to a value (*exponent*).

Example:

```
double x = pow(y, 32); // sets x to y raised to the 32nd power
```

double sqrt(x)
Returns the square root of a number.

Example:

```
double a = sqrt(1138); // approximately 33.73425674438
```

double sin(rad)

Returns the sine of an angle specified in radians.

Example:

```
double sine = sin(2); // approximately 0.90929737091
```

double cos(rad)

Returns the cosine of an angle specified in radians.

Example:

```
double cosine = cos(2); // approximately -0.41614685058
```

double tan(rad)

Returns the tangent of an angle specified in radians.

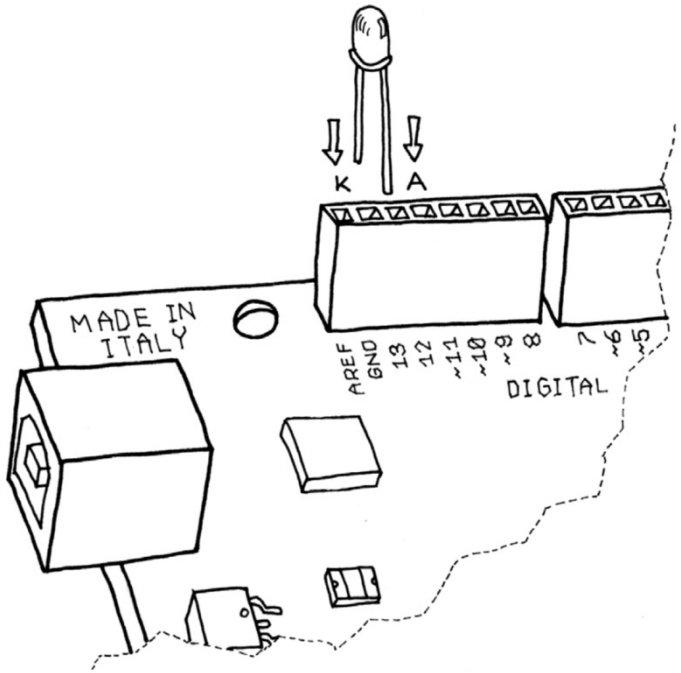
Example:

```
double tangent = tan(2); // approximately -2.18503975868
```

A Tour of Open-Source Activities in IoT

Blinking an LED

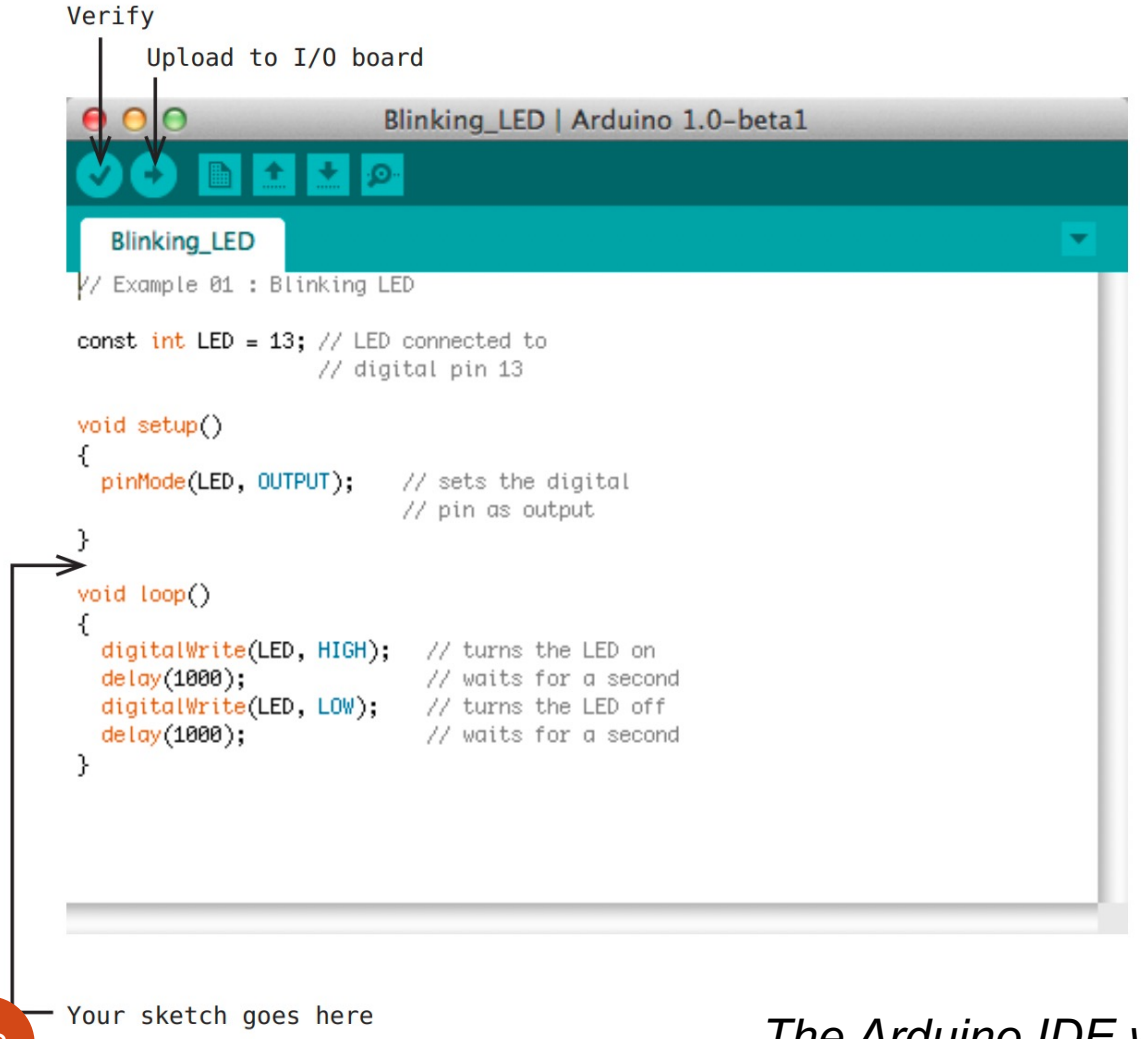
- The LED blinking sketch is the first program that you should run to test whether your Arduino board is working and is configured correctly. A light-emitting diode (LED) is a small electronic component that's a bit like a light bulb, but is more efficient and requires lower voltages to operate.



Connecting an LED to Arduino

A Tour of Open-Source Activities in IoT

Blinking an LED



The Arduino IDE with your first sketch loaded

A Tour of Open-Source Activities in IoT

```
// Example 01 : Blinking LED

const int LED = 13; // LED connected to
                    // digital pin 13

void setup()
{
  pinMode(LED, OUTPUT);    // sets the digital
                           // pin as output
}

void loop()
{
  digitalWrite(LED, HIGH);   // turns the LED on
  delay(1000);               // waits for a second
  digitalWrite(LED, LOW);    // turns the LED off
  delay(1000);               // waits for a second
}
```

A Tour of Open-Source Activities in IoT

The Code, Step by Step

```
// Example 01 : Blinking LED
```

A comment is a useful way for us to write little notes. The preceding title comment just reminds us that this program, Example 01, blinks an LED.

```
const int LED = 13;  // LED connected to
                     // digital pin 13
```

const int means that LED is an integer number that can't be changed (i.e. a constant) whose value is set to 13. It's like an automatic search and replace for your code; in this case, it's telling Arduino to write the number 13 every time the word LED appears. We are using this command to specify that the LED we're blinking is connected to the Arduino pin 13.

The Code, Step by Step

```
void setup()
```

This line tells Arduino that the next block of code will be called *setup()*.

```
{
```

With this opening curly bracket, a block of code begins.

```
pinMode(LED, OUTPUT); // sets the digital
                      // pin as output
```

Finally, a really interesting instruction. *pinMode* tells Arduino how to configure a certain pin. Digital pins can be used either as INPUT or OUTPUT. In this case, we need an output pin to control our LED, so we place the number of the pin and its mode inside the parentheses.
pinMode is a function, and the words (or numbers) specified inside the parentheses are **arguments**. INPUT and OUTPUT are constants in the Arduino language. (Like variables, constants are assigned values, except that constant values are predefined and never change.)

```
}
```

This closing curly bracket signifies the end of the *setup()* function.

The Code, Step by Step

```
void loop()
{
```

loop() is where you specify the main behaviour of your interactive device. It will be repeated over and over again until you switch the board off.

```
digitalWrite(LED, HIGH);   // turns the LED on
```

As the comment says, *digitalWrite()* is able to turn on (or off) any pin that has been configured as an OUTPUT. The first argument (in this case, *LED*) specifies which pin should be turned on or off (remember that *LED* is a constant value that refers to pin 13, so this is the pin that's switched). The second argument can turn the pin on (HIGH) or off (LOW).

The Code, Step by Step

```
delay(1000);      // waits for a second
```

Arduino has a very basic structure. Therefore, if you want things to happen with a certain regularity, you tell it to sit quietly and do nothing until it is time to go to the next step. *delay()* basically makes the processor sit there and do nothing for the amount of milliseconds that you pass as an argument. Milliseconds are thousandths of seconds; therefore, 1000 milliseconds equals 1 second. So the LED stays on for one second here.

```
digitalWrite(LED, LOW);     // turns the LED off
```

This instruction now turns off the LED that we previously turned on. Why do we use HIGH and LOW? Well, it's an old convention in digital electronics. HIGH means that the pin is on, and in the case of Arduino, it will be set at 5 V. LOW means 0 V. You can also replace these arguments mentally with ON and OFF.

A Tour of Open-Source Activities in IoT

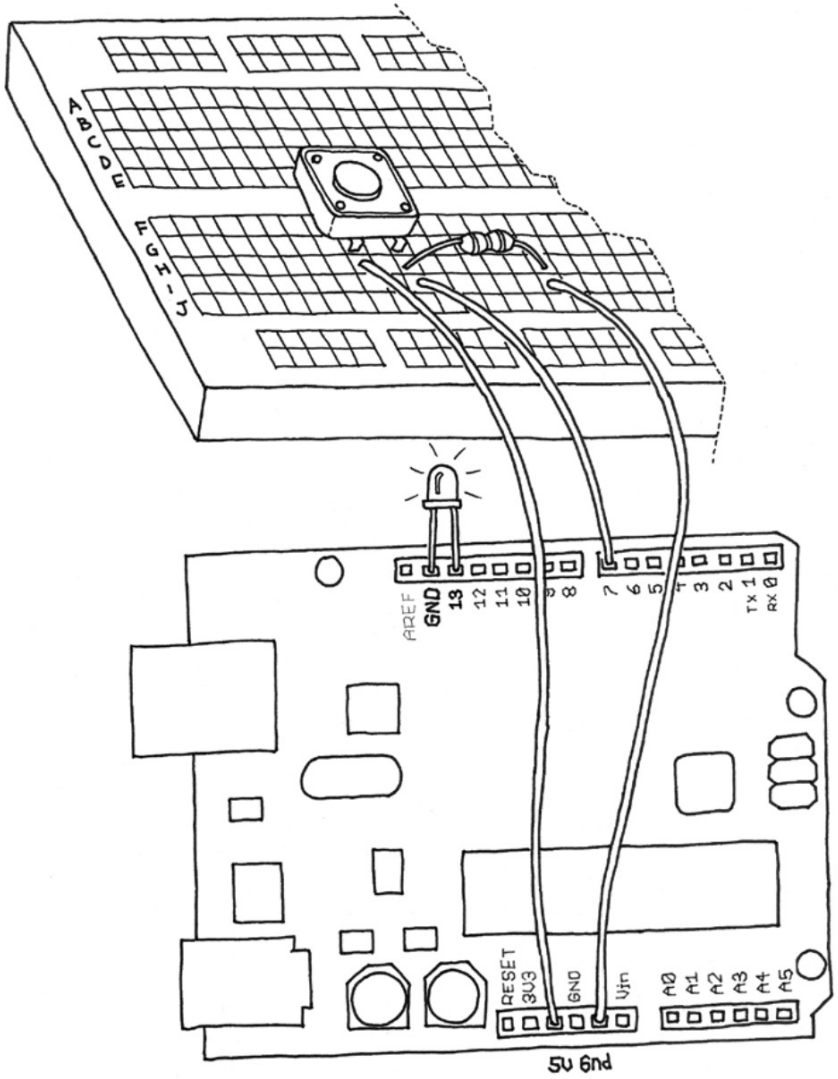
The Code, Step by Step

To sum up, this program does this:

- Turns pin 13 into an output (just once at the beginning)
- Enters a loop
- Switches on the LED connected to pin 13
- Waits for a second
- Switches off the LED connected to pin 13
- Waits for a second
- Goes back to beginning of the loop

A Tour of Open-Source Activities in IoT

Using a Pushbutton to Control the LED



Hooking up a pushbutton

A Tour of Open-Source Activities in IoT

Using a Pushbutton to Control the LED

```
// Example 02: Turn on LED while the button is pressed

const int LED = 13;   // the pin for the LED
const int BUTTON = 7; // the input pin where the
                      // pushbutton is connected
int val = 0;          // val will be used to store the state
                      // of the input pin

void setup() {
  pinMode(LED, OUTPUT);   // tell Arduino LED is an output
  pinMode(BUTTON, INPUT); // and BUTTON is an input
}

void loop(){
  val = digitalRead(BUTTON); // read input value and store it

  // check whether the input is HIGH (button pressed)
  if (val == HIGH) {
    digitalWrite(LED, HIGH); // turn LED ON
  } else {
    digitalWrite(LED, LOW);
  }
}
```

MicroPython Programming with ESP32

Integrated Development Environment

uPyCraft IDE

There are different programs you can use to program the ESP32 and ESP8266 boards. To program the ESP32 or ESP8266 boards with MicroPython firmware, we recommend using **uPyCraft IDE** because it is simple, intuitive to use and works great with the ESP boards. However, if you are used to other MicroPython programming environment, you can stick with it.

uPyCraft IDE runs in any major operating system. We have instructions for installing uPyCraft IDE in Window, Mac OS X and Linux Ubuntu. Install uPyCraft IDE in your computer by following one the installation procedures we describe in the following Units.

Integrated Development Environment

Alternative – Thonny IDE

We've experimented with several IDEs to program the ESP32 and ESP8266 boards using MicroPython, and uPyCraft IDE worked fine for us. However, some users reported some trouble using uPyCraft IDE. So, we provide an alternative IDE to program your ESP32/ESP8266: **Thonny IDE**.



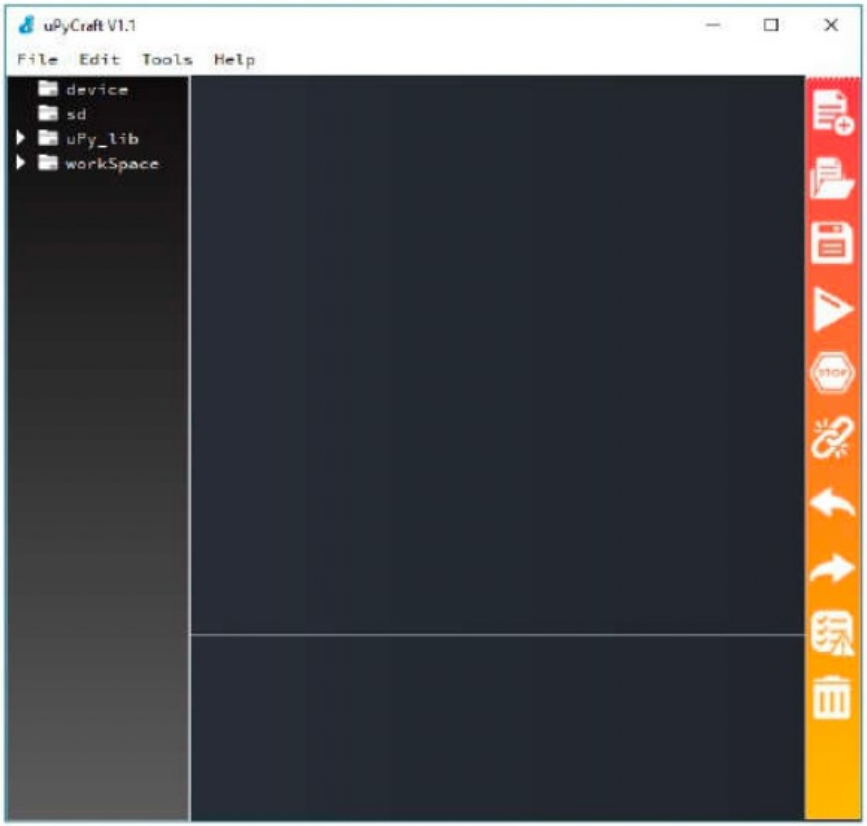


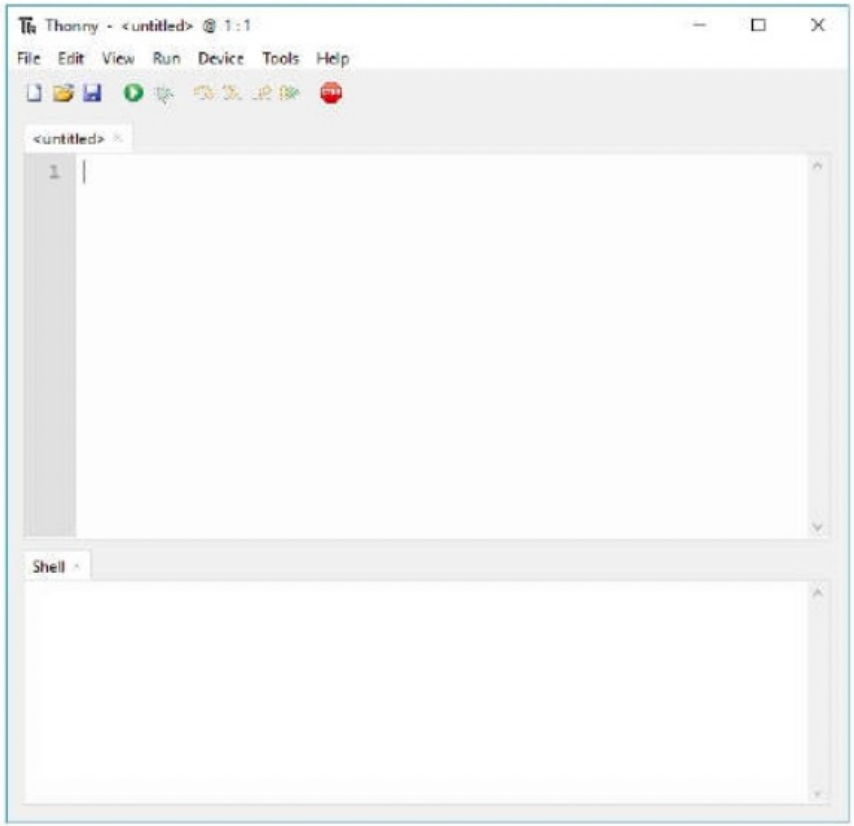
Integrated Development Environment

The main differences between uPyCraft IDE and Thonny IDE:

- uPyCraft is an IDE for MicroPython, whereas Thonny is an IDE for Python with MicroPython support;
- uPyCraft IDE prints all the debug information in the debugging window which can be confusing. Thonny IDE has a layer on top that doesn't show all those errors, so it's a bit more user friendly;
- uPyCraft IDE has more features dedicated to program ESP8266/ESP32 with MicroPython, but it's harder to install on some Linux computers. Thonny IDE is easier to install and compatible with all operating systems;
- uPyCraft provides a better way to edit and delete files saved on your boards. On the other hand, Thonny IDE has a built-in feature to upload files as *boot.py* or *main.py*. That feature makes it easier and faster to upload new scripts.

Integrated Development Environment



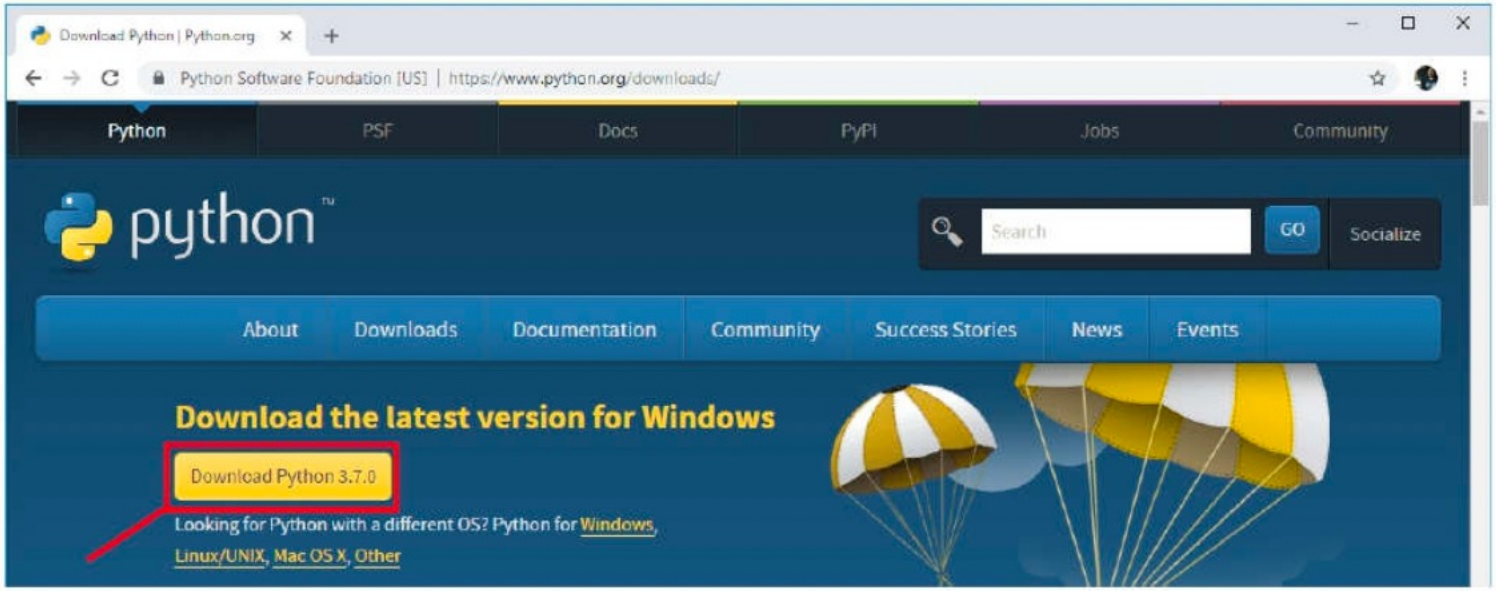


Installing uPyCraft IDE (Windows)

Installing Python 3.7.X

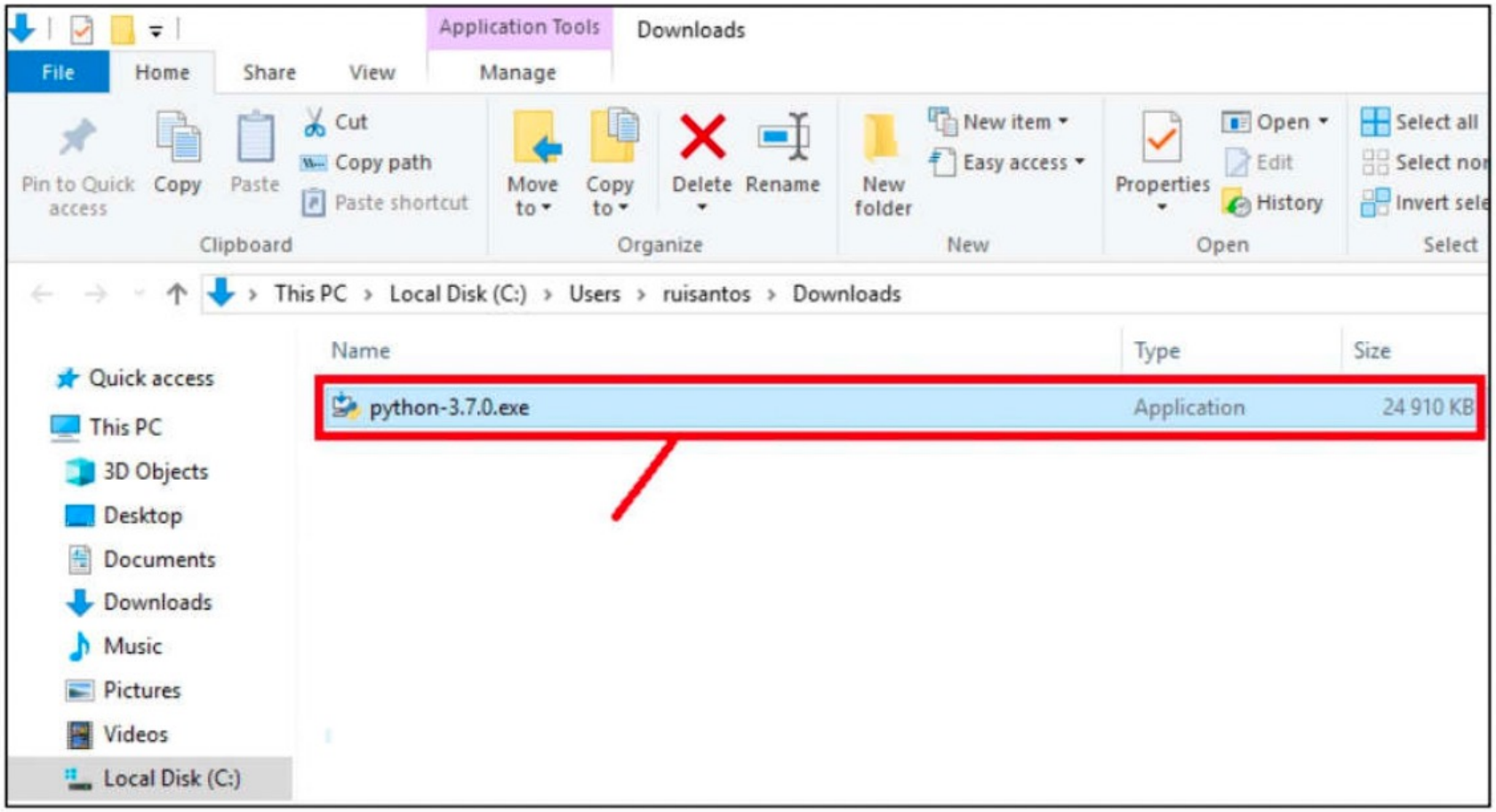
Before installing uPyCraft IDE, make sure you have the latest version of **Python 3.7.X** installed in your computer. Follow the next installation instructions:

1. Go to the Python Downloads page: www.python.org/downloads and download the installation file.



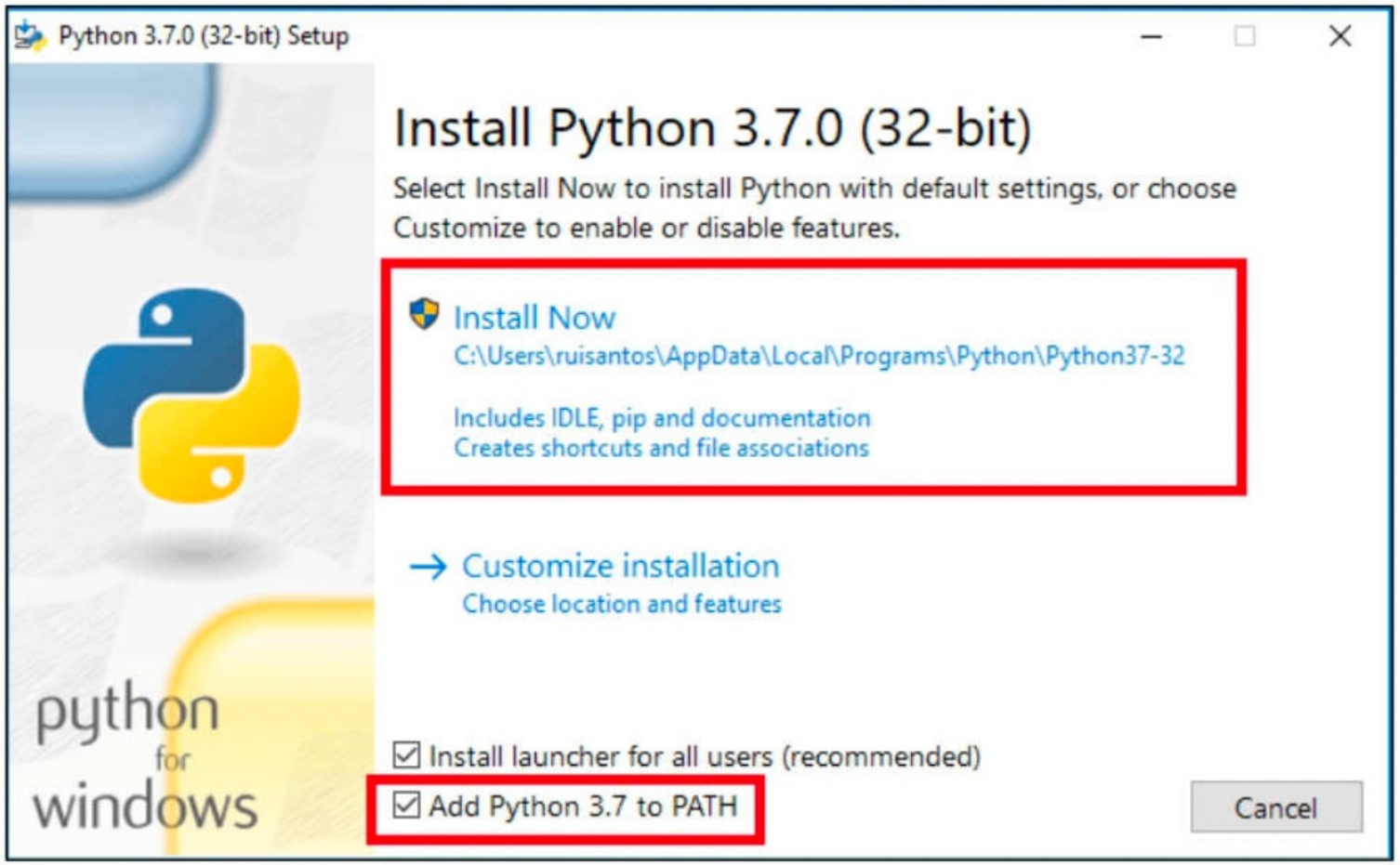
Integrated Development Environment

2. After a few seconds, you should have a file called python-3.7.X.exe file in your computer. Double-click the file to open it.



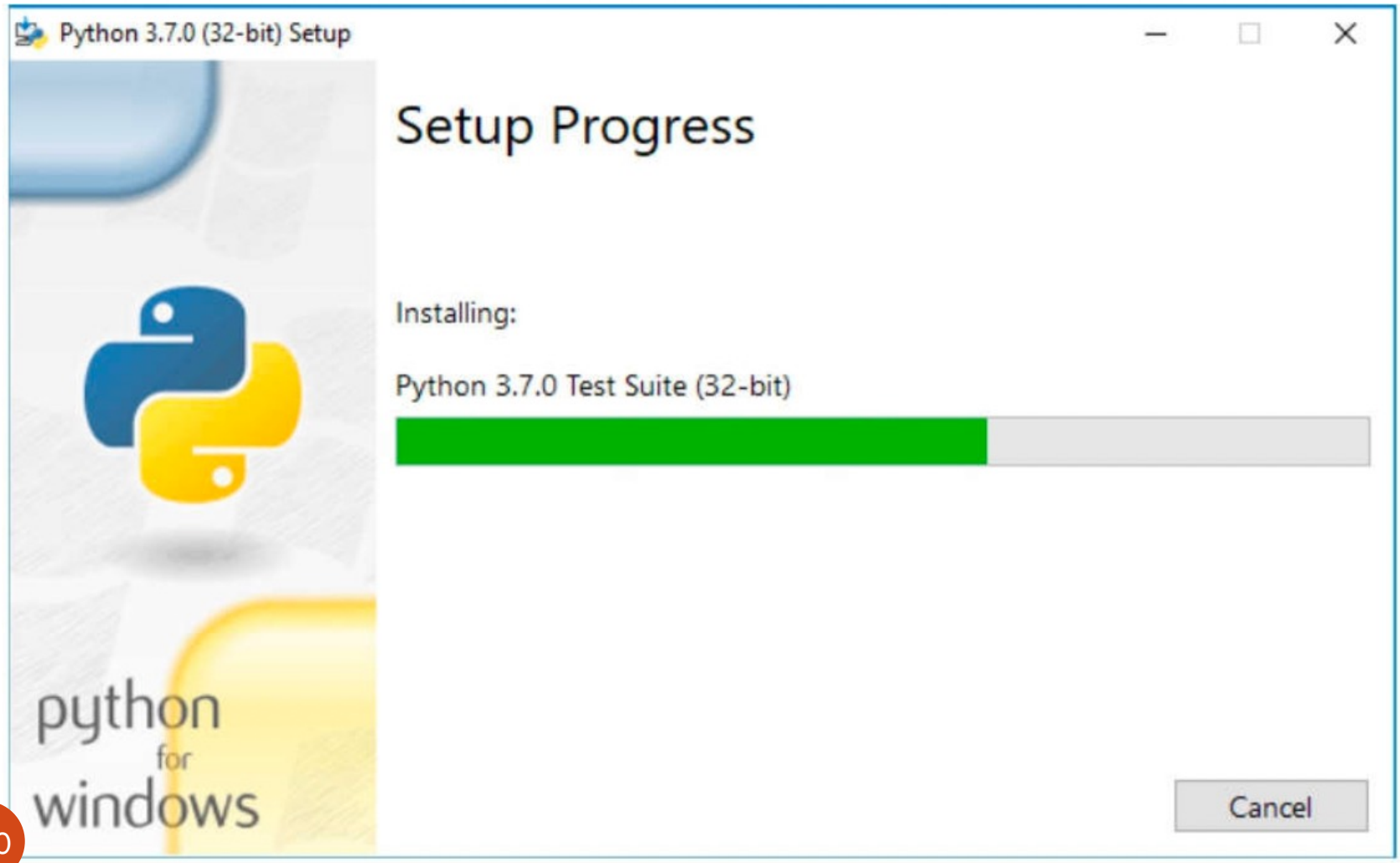
Integrated Development Environment

3. Enable the option at the bottom "**Add Python 3.7 to PATH**". Then, press the "**Install Now**" button:



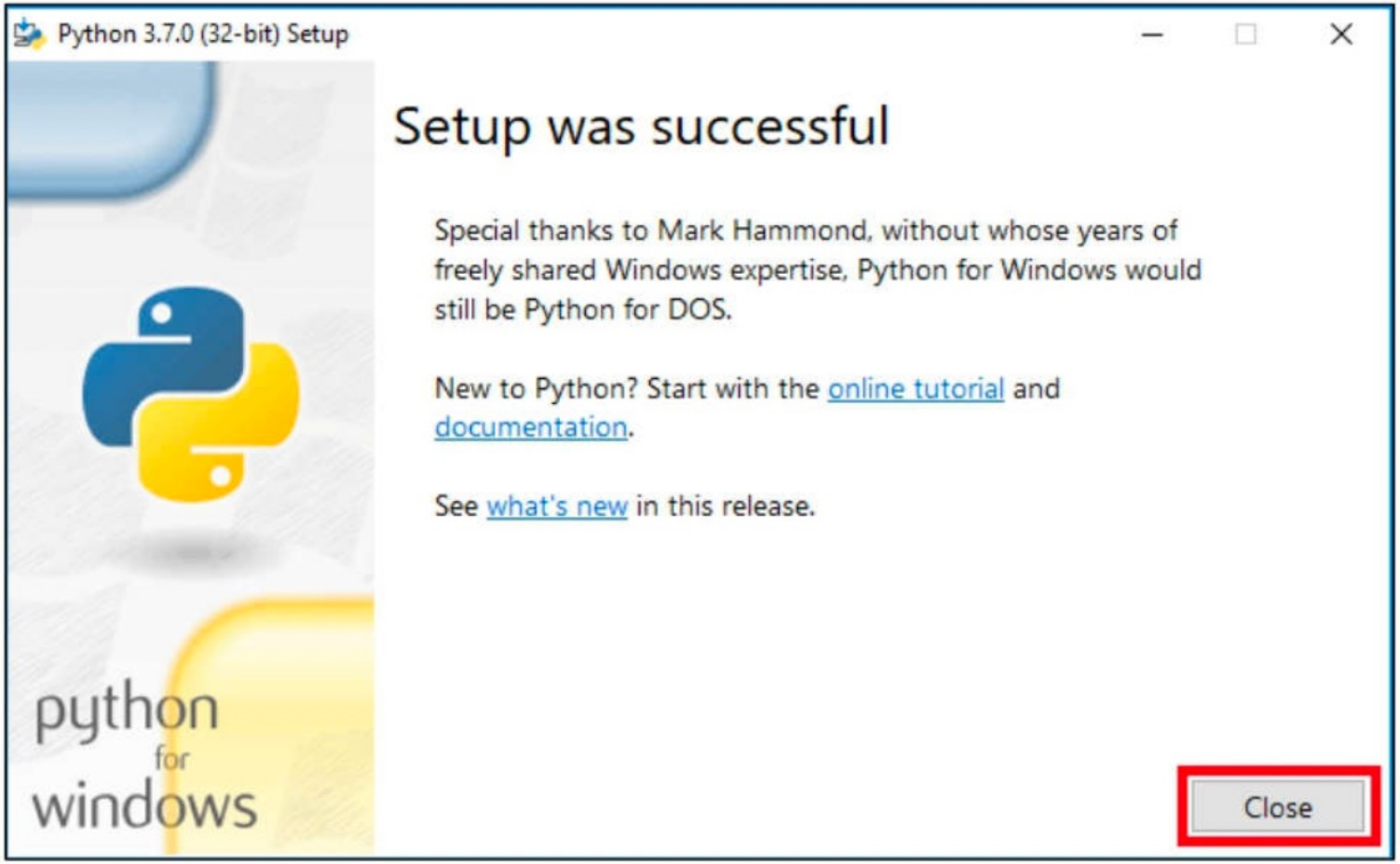
Integrated Development Environment

4. Wait a few seconds while the software completes the installation process.



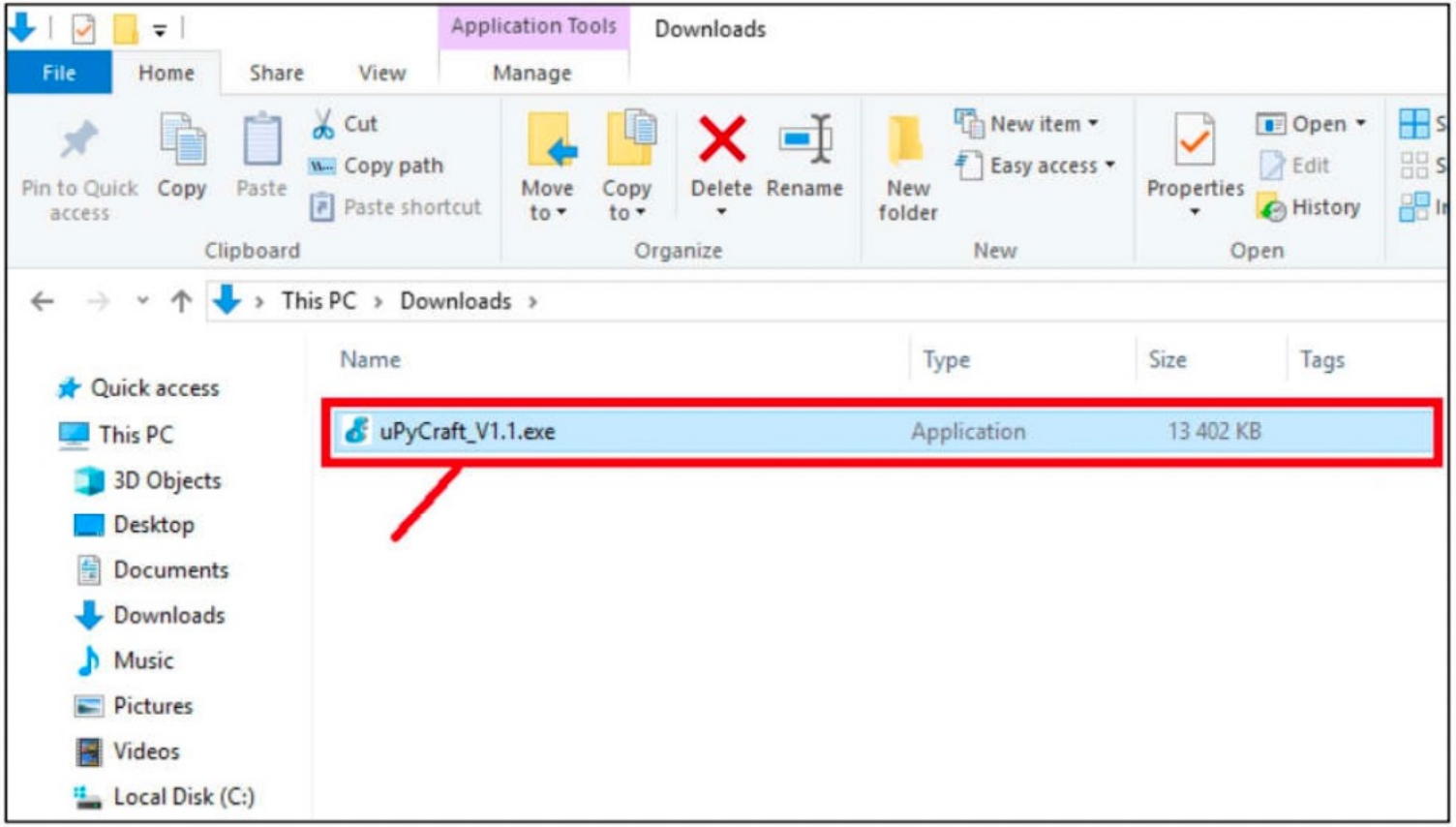
Integrated Development Environment

5. When it's done, you should see the message "**Setup was successful**" and you can close that window.



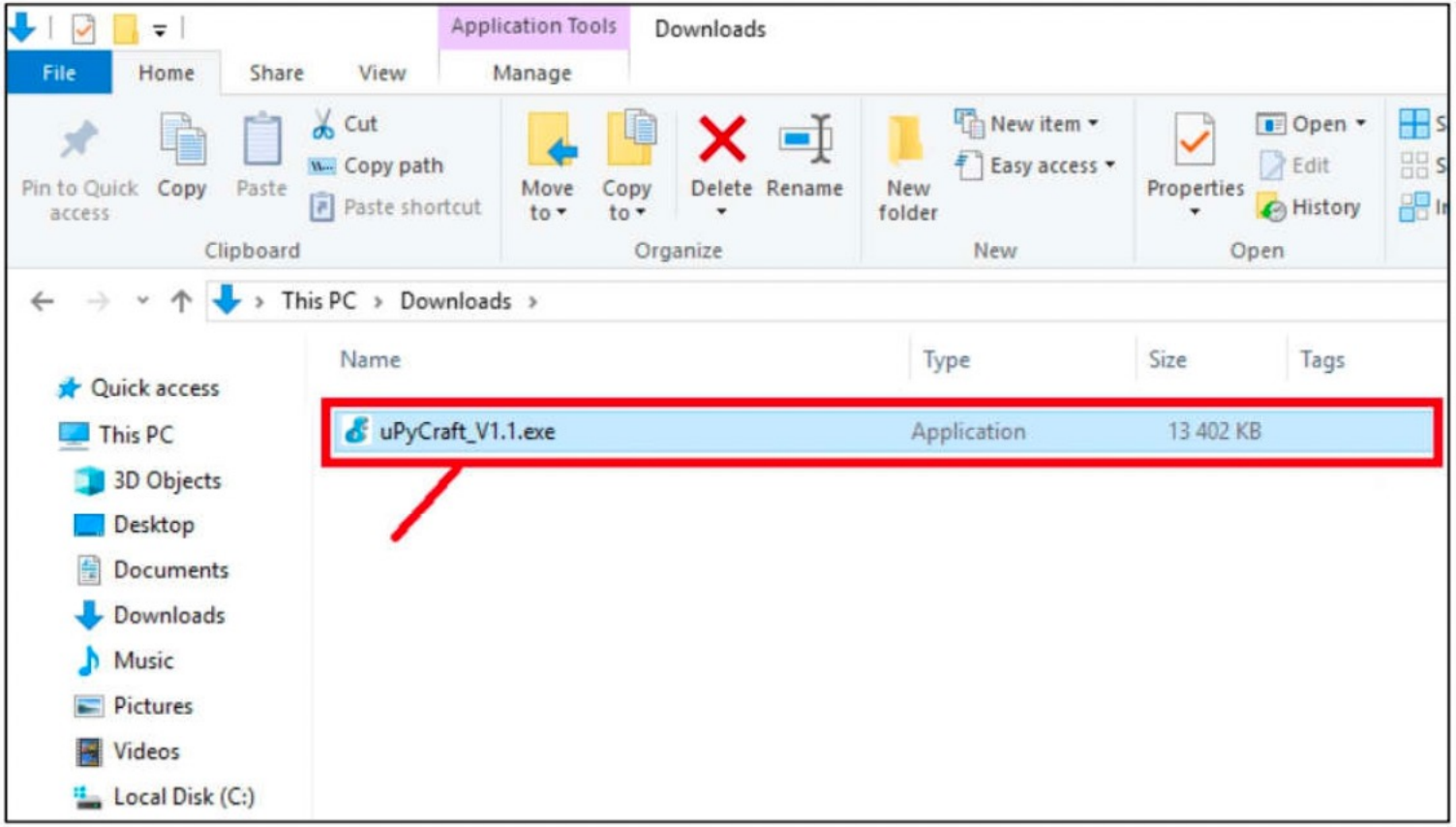
Integrated Development Environment

https://randomnerdtutorials.com/uPyCraftWindows



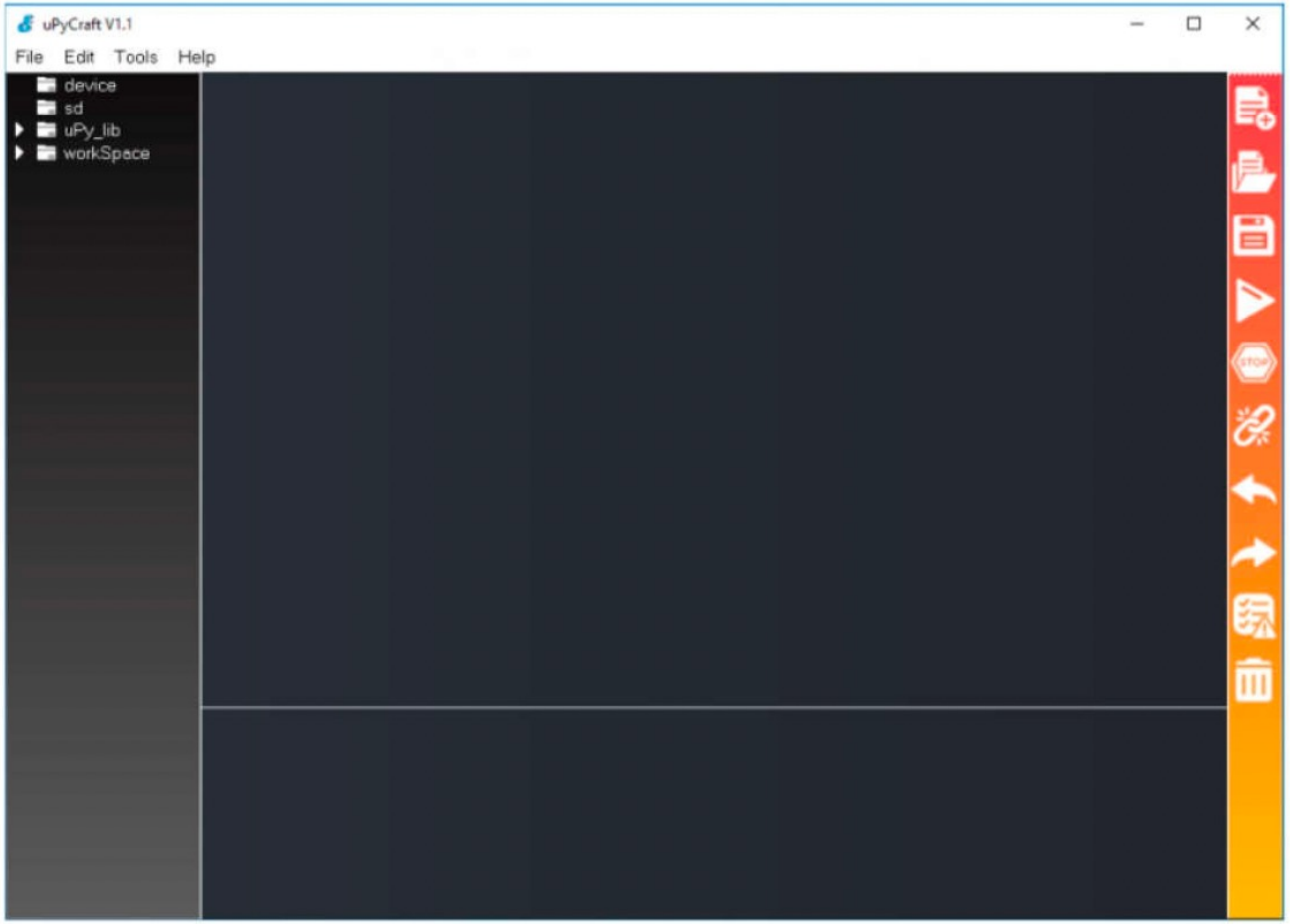
Integrated Development Environment

https://randomnerdtutorials.com/uPyCraftWindows

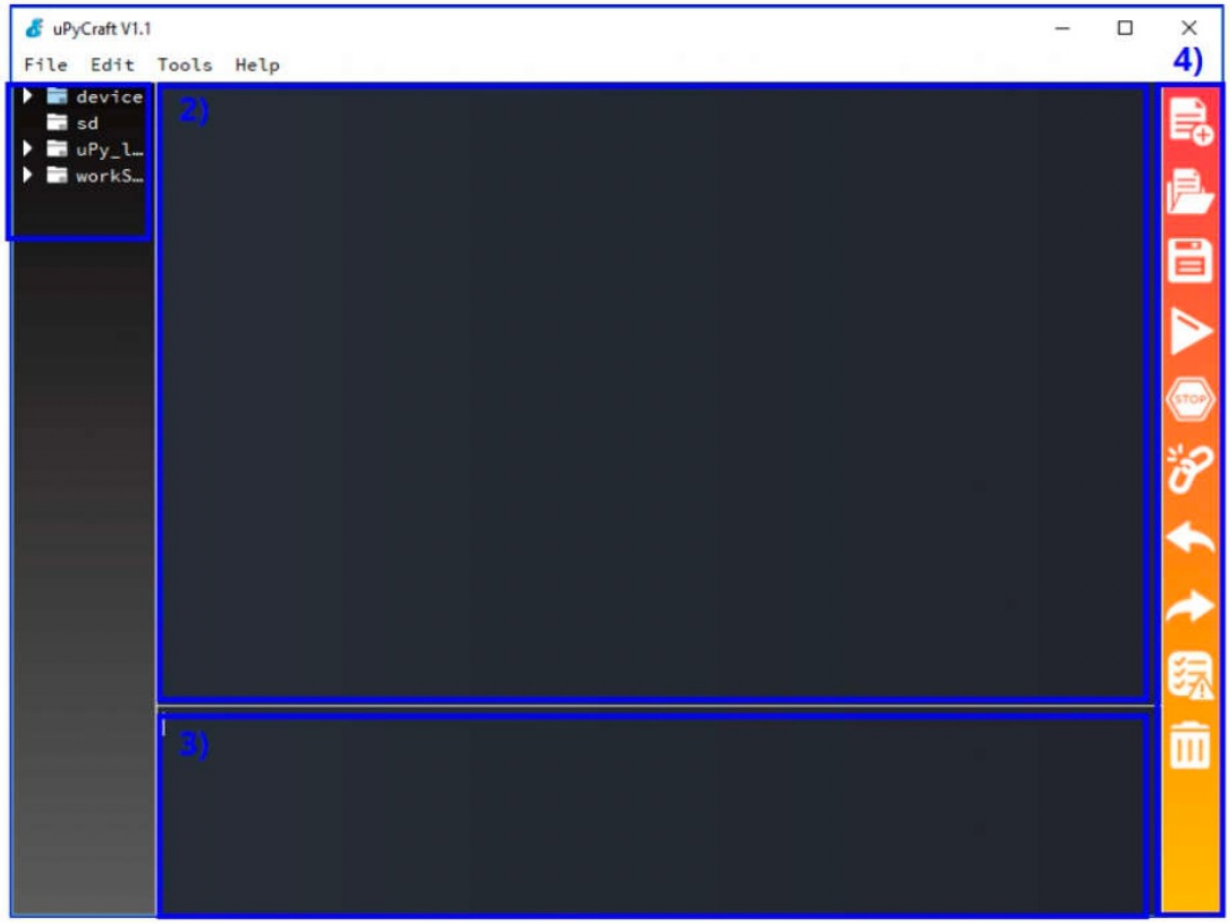


Integrated Development Environment

Double-click the downloaded file. A new window opens with the uPyCraft IDE software:



uPyCraft IDE Overview



1. Folder and files
2. Editor
3. MicroPython Shell/Terminal
4. Tools

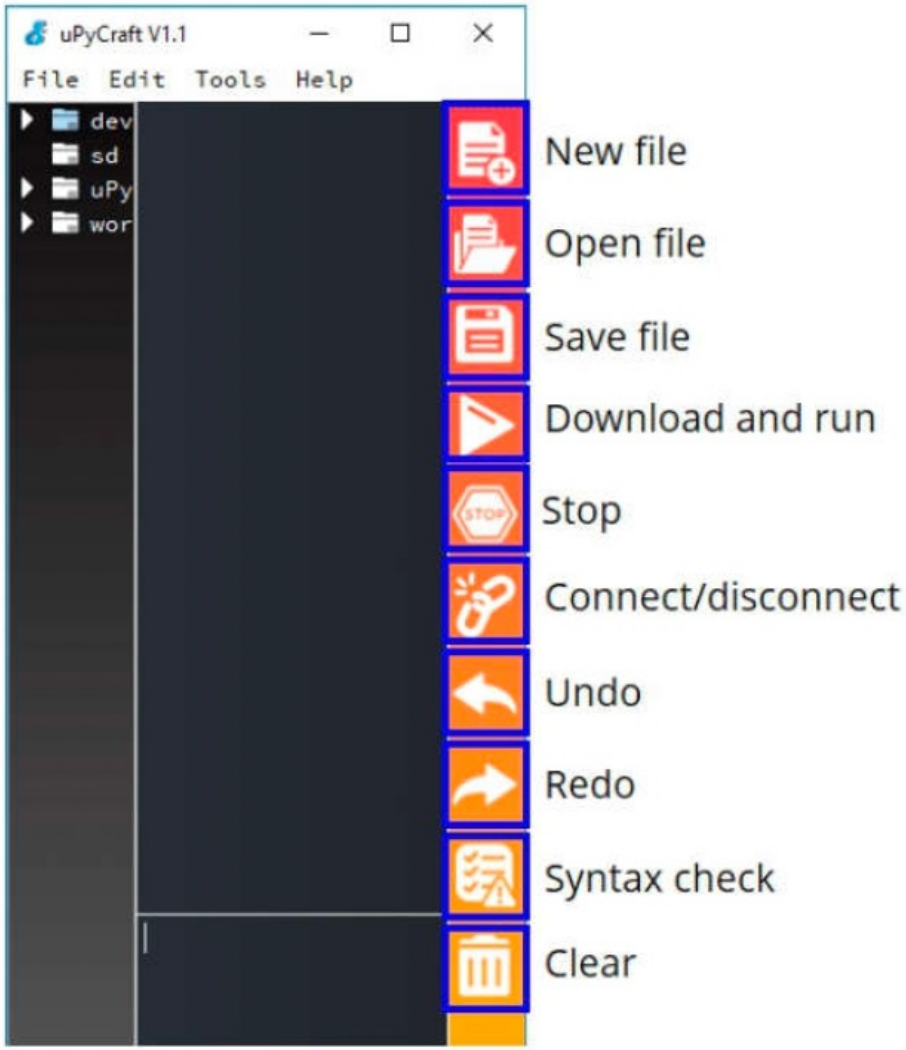
uPyCraft IDE Overview

MicroPython Shell/terminal

On the MicroPython Shell you can type commands to be executed immediately by your ESP board without the need to upload new files. The terminal also provides information about the state of an executing program, shows errors related with upload, syntax errors, prints messages, etc...

uPyCraft IDE Overview

Tools



uPyCraft IDE Overview

Tools

- **New file**: creates a new file on the Editor;
- **Open file**: opens a file from your computer;
- **Save file**: saves a file;
- **Download and run**: upload the code to your board and execute the code;
- **Stop**: stop the execution of the code – it's the same as entering CRTL+C on the Shell to stop all scripts from running;
- **Connect/Disconnect**: connect or disconnect to your board via Serial. You must select the serial port first in **Tools ▸ Serial**;
- **Undo**: undo last change in the code Editor;

uPyCraft IDE Overview

Tools

- **Redo**: redo last change in the code Editor;
- **Syntax check**: checks the syntax of your code;
- **Clear**: clear the Shell/terminal window messages.

uPyCraft IDE Overview

Establishing a communication with the board

After having the MicroPython firmware installed on your board and having the board connected to your computer through a USB cable, follow the next steps:

1. Go to **Tools** ▸ **Board** and select the board you're using.

2. Go to **Tools** ▸ **Port** and select the COM port your ESP is connected to.

3. Press the **Connect** button to establish a serial communication with your board.



Connect/disconnect

uPyCraft IDE Overview

4. The **>>>** should appear in the Shell window after a successful connection with your board. You can type the `print` command to test if it's working:

```
>>> print('Hello')
Hello
>>>
```

uPyCraft IDE Overview

It should print the "**Hello**" message. Only if you see that message, you can continue with this tutorial. Otherwise, make sure you have established a serial communication with your board or that you've successfully flashed the MicroPython firmware on your board.



uPyCraft IDE Overview

Creating the *main.py* file on your board

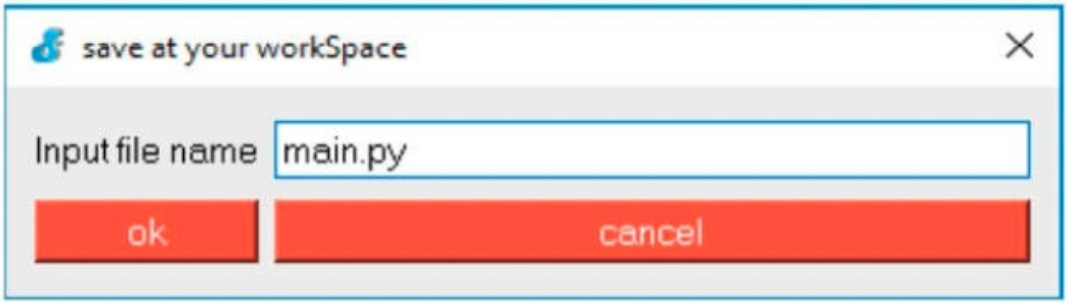
1. Press the "**New file**" button to create a new file.



2. Press the "**Save file**" button to save the file in your computer.

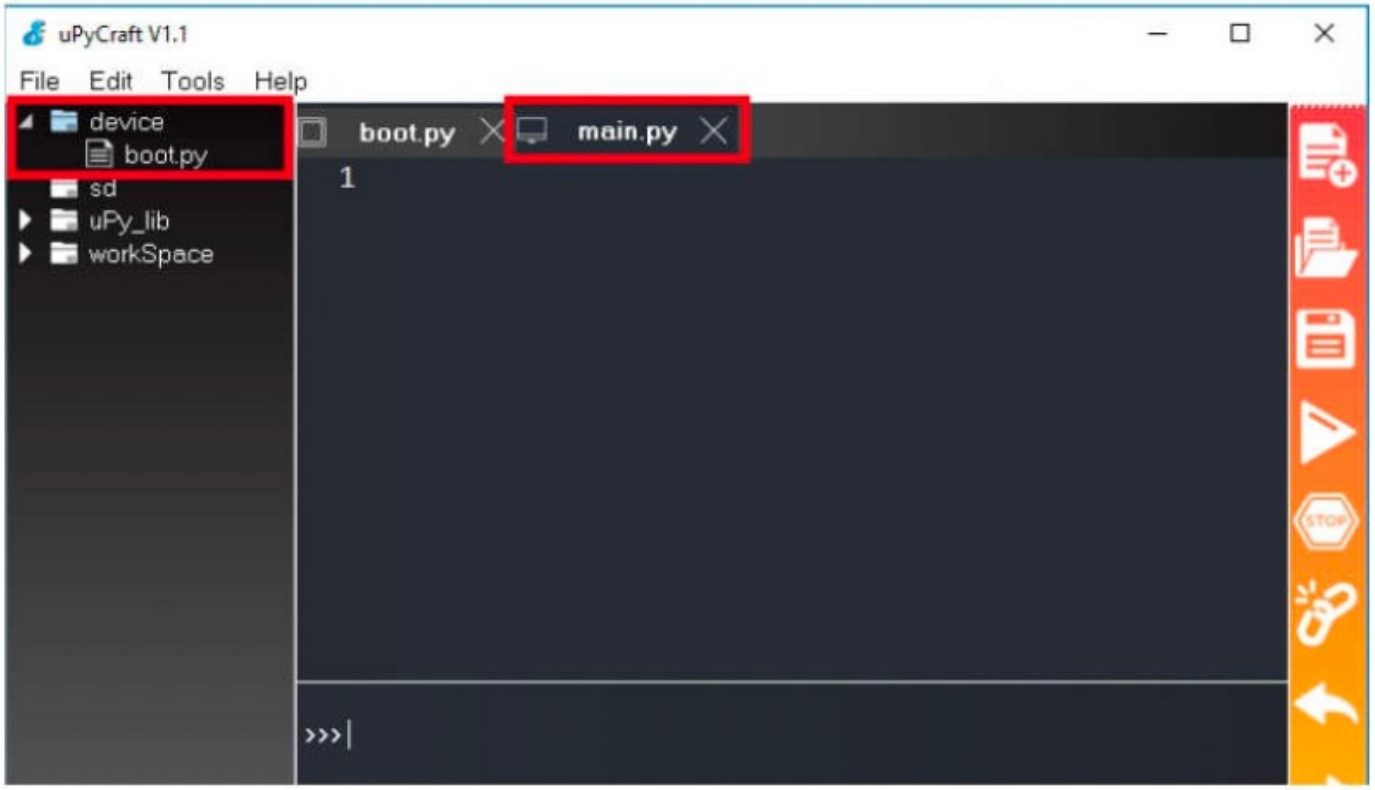


3. A new window opens, name your file ***main.py*** and save it in your computer:



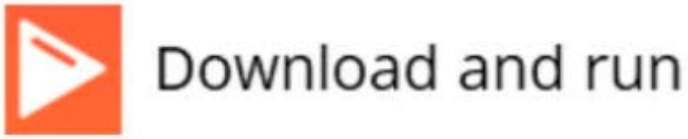
uPyCraft IDE Overview

4. After that, you should see the following in your uPyCraft IDE (the *boot.py* file in your device and a new tab with the *main.py* file):

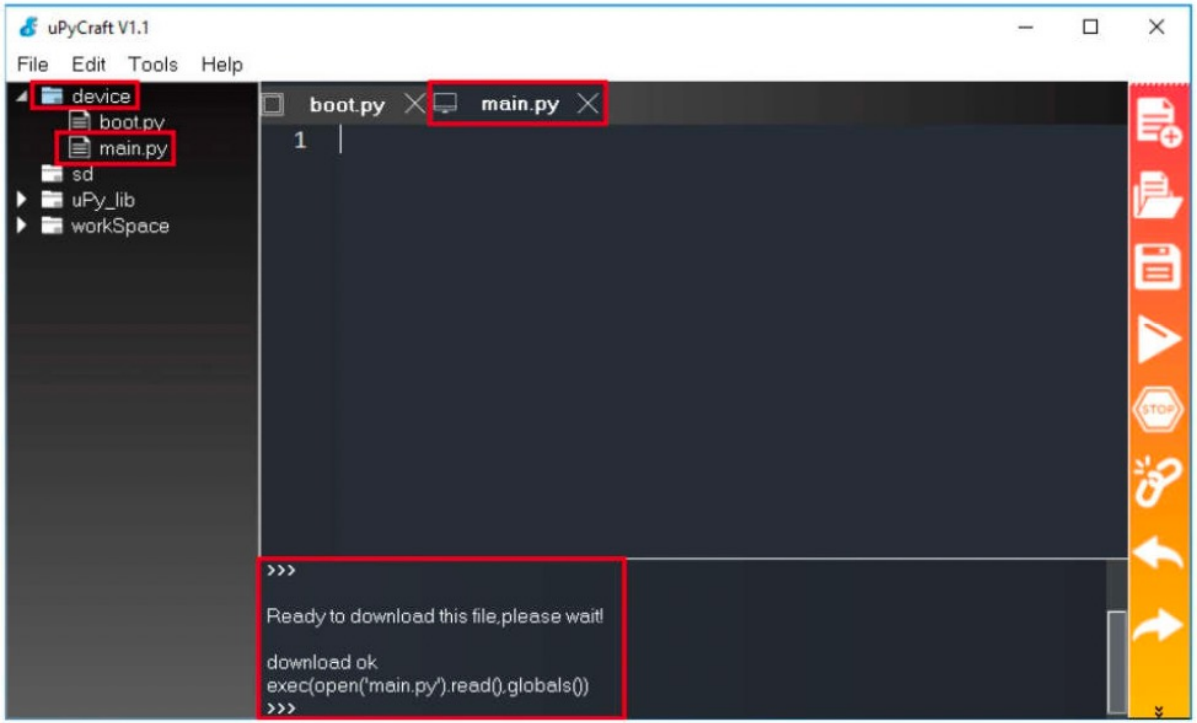


uPyCraft IDE Overview

5. Click the "**Download and run**" button to upload the file to your ESP board:



6. The **device** directory should load the ***main.py*** file.



uPyCraft IDE Overview

Uploading the blink LED script

1. Copy the following code to the Editor on the ***main.py*** file:

```
from machine import Pin
from time import sleep

led = Pin(2, Pin.OUT)

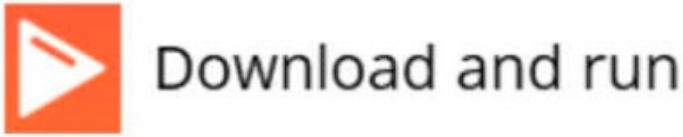
while True:
  led.value(not led.value())
  sleep(0.5)
```

uPyCraft IDE Overview

2. Press the "**Stop**" button to stop any script from running in your board:

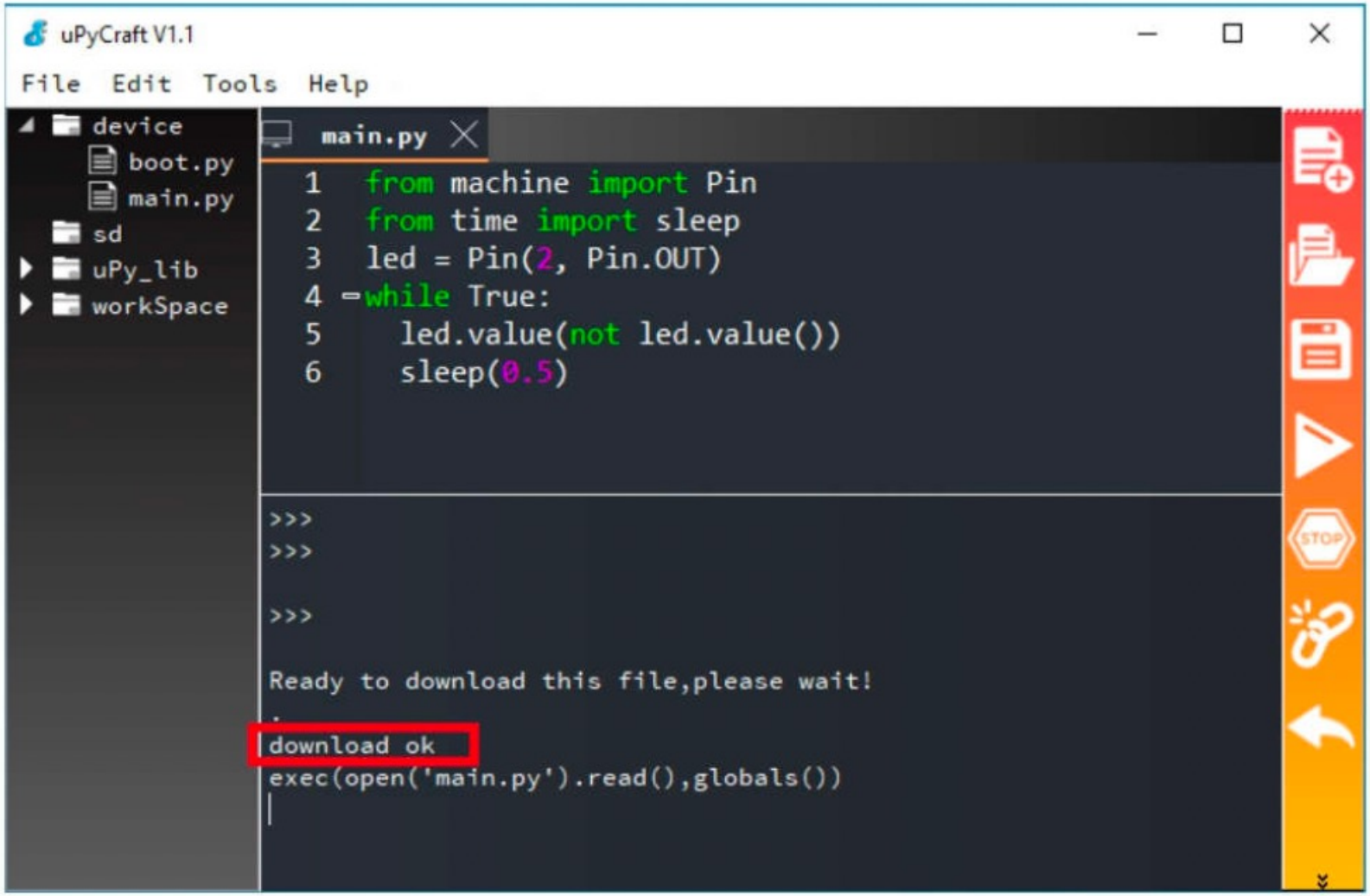


3. Click the "**Download and Run button**" to upload the script to the board:



uPyCraft IDE Overview

4. You should see a message saying "**download ok**" in the Shell window.



MicroPython Programming Basics

Programming in MicroPython is very similar to programming in Python: all language features of Python are also in MicroPython, apart from a few exceptions. Because microcontrollers and embedded systems are much more limited than our computers, MicroPython does not come with the full standard library by default.

If you already know how to program in Python, programming in MicroPython is the same. You just need to bear in mind that MicroPython is used for constrained devices. Therefore, you must keep your code as simple as possible.

MicroPython Programming Basics

This section explains the basics of Python programming language syntax that also apply to MicroPython, like:

- Mathematical operators
- Relational operators
- Data types
- print() function
- Comments

MicroPython Programming Basics

This section explains the basics of Python programming language syntax that also apply to MicroPython, like:

- Mathematical operators
- Relational operators
- Data types
- print() function
- Comments
- Conditional statements
- While and for loops
- User defined functions
- Classes and objects
- Modules

Mathematical Operators

MicroPython can perform mathematical operations. The following table shows the mathematical operators supported:

Operator	Mathematical Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
//	Division, discarding the decimal point
%	Remainder after division

MicroPython Programming Basics

In the Shell, try several operations to see how it works. For example:

```
>>> 2+2*9-3
17
>>> 28594/2312
12.36765
>>> 214522236/7.5
2.860297e+07
>>> 23//2
11
>>> 25%3
1
```

You can perform other mathematical operations if you import the `math` module, like square root, trigonometric functions, logarithm, exponentiation, etc.

Relational Operators

You can make comparisons using relational operators. These compare the values on either sides and show the relation between them.

Operator	Description
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

MicroPython Programming Basics

Try several comparisons and test the result:

```
>>> 2 == 3
False
>>> 4 == 4
True
>>> 3 > 2
True
>>> 489808234 != 2223
True
>>> 4.5 >= 4.5
True
```

Assigning Values to Variables

In Python, you don't need to declare what type each variable is. If you're used to program your boards using Arduino IDE, you know that you need to declare the type of a variable when creating a new variable. There isn't such thing in Python.

Variables are simply a storage placeholder for values: number or text. To assign a value to a variable you use the equal sign (=), with the variable name on the left and the value on the right.

MicroPython Programming Basics

For example, to create a variable to hold the GPIO number where an LED is connected to, you can simply type the following:

```
led_pin = 23
```

In the Arduino IDE, you would have something like:

```
const int led_pin = 23;
```

As you can see, Python is much simpler than programming in C (in Arduino IDE).

MicroPython Programming Basics

Data Types

Variables can store several types of values, not just whole numbers. That's where data types come in. A *data type* is a classification of a value that tells what operations can be done with the value and how it should be stored.

The following table shows the data types we'll use more often in our projects.

Data type	Description
int (Int)	Integer (whole number)
float (Float)	Number with a decimal point
str (String)	Set of characters between quotation marks
bool (Boolean)	True or False

MicroPython Programming Basics

Let's create variables with different data types:

```
>>> a = 6
>>> b = 95.32
>>> c = 'Hello World!'
>>> d = True
```

- The first value assigned to `a`, is an **integer**, which is a whole number.
- The `b` variable contains a **float** value, which is a number with a decimal.
- The third value, `'Hello World! '`, is a **string**, which is a series of characters. A string must be put inside single `'Hello World!'` or double quotation `"Hello World!"` marks.
- Finally, `d` is a **Boolean**, which is a type that can only take either `True` or `False`.

MicroPython Programming Basics

There is a function to check the data type of a variable: the `type()` function. This function accepts as argument the variable you want to check the data type.

```
type(variable)
```

For example, after declaring the variables in the previous example `a`, `b`, `c`, and `d`, you can check its data type. For example:

```
>>> type(a)
```

Returns:

```
<class 'int'>
```

MicroPython Programming Basics

This tells that `a` is an int (integer). Experiment with the other variables and you should get:

```
>>> type(b)
<class 'float'>
>>> type(c)
<class 'str'>
>>> type(d)
<class 'bool'>
```

print() Function

The `print()` function prints the message between parentheses into the Shell.

This is useful to debug the code and keep track of what's going on. For example:

```
>>> print('LED is on')
LED is on
```

MicroPython Programming Basics

Comments

Comments in Python start with the hash character # and continue to the end of the line. A comment is useful to add "notes" in your program or to tell anyone who reads the program what the script does. This doesn't add any functionality to your program. For example:

```
# This is just a comment
```

Because in MicroPython we are working under constrained conditions, there are occasions in which you should avoid adding comments to save space on the flash memory.

Conditional Statements

To write useful programs, you'll probably need to perform different actions depending on whether a certain condition is True or False. We're talking about conditional statements. They have the following structure:

```
if <expr1>:
  <statement1>
elif <expr2>:
  <statement2>
elif <expr3>:
  <statement3>
(...)
else:
  <statementn>
```

While and For loops

Loops allow you to execute a block of code multiple times for as long as a condition is met. There are two kinds of loops: while and for loops.

For example, you can print all numbers from 1 to 10 with a while loop:

```
number = 1
while number <= 10:
  print(number)
  number = number + 1
```

The code that belongs to the `while` loop, indicated by the indentation, is executed as long as the value in the variable `number` is less than or equal to (<=) 10. In every loop, the current number is printed and then we add 1 to it.

MicroPython Programming Basics

You can also print numbers from 1 to 10 using a for loop, like this:

```
number = 1
for number in range(1, 11):
  print(number)
```

The `for` loop is executed as long as the value in the `number` variable is within the range of 1 and 11. The `range()` function automatically assigns the next value to the `number` variable, until 1 below the final number you specify.

You should use a `for` loop when you want to repeat a block of code a certain number of times. Use a while loop when you want to repeat code until a certain condition is no longer met. In some situations, you can use either one, oftentimes one is more suitable than the other.

User-defined Functions

To define a new function, you use the word `def` followed by the name you want to give the function and a set of parentheses (and arguments inside, if necessary). After the parentheses you add a colon : and then tell the function what instructions to perform. The statements should be indented with 2 spaces (in MicroPython):

```
def my_function(<arg1>, <arg2>, ...):
  <statement>
  (...)
  return
```

MicroPython Programming Basics

For example, a function that converts the temperature in Celsius to Fahrenheit could be as follows:

```
def celsius_to_fahrenheit(temp_celsius):
  temp_fahrenheit = temp_celsius * (9/5) + 32
  return temp_fahrenheit
```

The `celsius_to_fahrenheit()` function accepts as argument a temperature in Celsius (`temp_celsius`). Then, it does the calculation to convert the temperature. Finally, it returns the temperature in Fahrenheit (`temp_fahrenheit`).

Modules

A **module** is a file that contains a set of classes and functions you can use in your code – you can also call it library. To access the classes and functions inside that code, you just need to import that module into your code.

You can create your own modules or use already created modules from the standard Python library. When it comes to MicroPython, it only comes with a small subset of the standard Python library, but it does come with a set of modules to control GPIOs, make networks connections and much more.

MicroPython Programming Basics

Importing modules/libraries is as simple as using:

```
import module_name
```

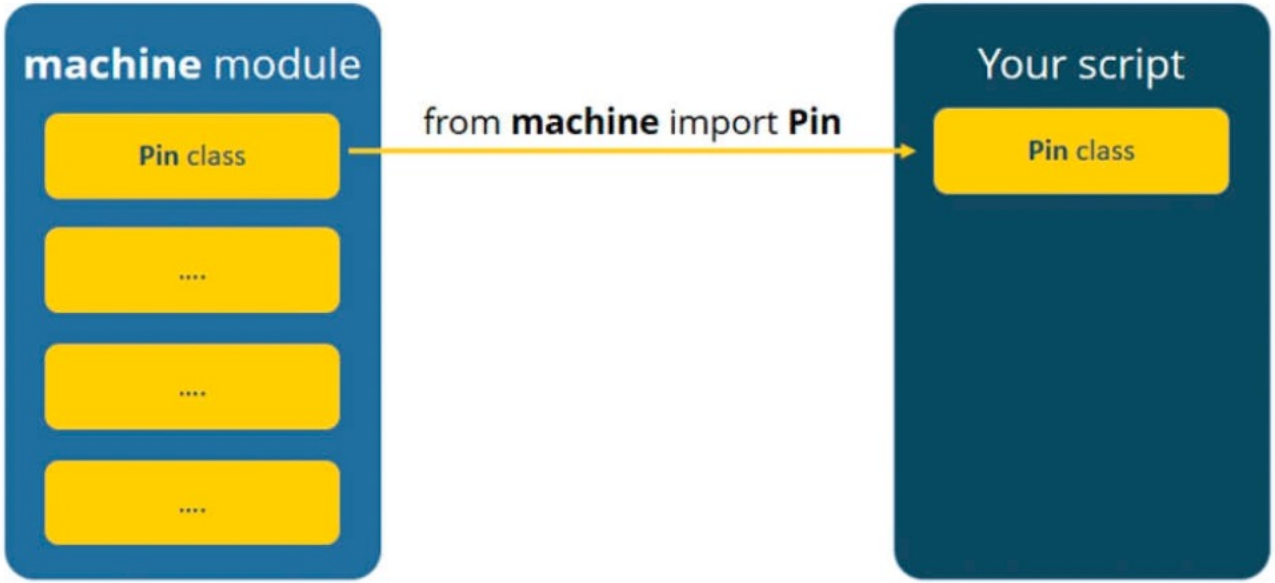
For example, to import the `machine` library that contains classes to control GPIOs, type the following:

```
import machine
```

MicroPython Programming Basics

In most programs you won't need all the classes from one module. You may just want to import a single class. For example, to import only the `Pin` class from the `machine` module:

```
from machine import Pin
```



MicroPython vs C++ (Arduino IDE)

If you are used to program electronics using the Arduino IDE, you'll find that MicroPython has a much simpler and user-friendly syntax. Let's just summarize some of the main differences between your Arduino sketches and scripts in MicroPython:

- You don't use semicolon ; at the end of a statement
- After Boolean expressions in conditional statements and loops, you use a colon **:**
- To define code blocks use indentation, instead of curly brackets **{}**

MicroPython Programming Basics

- When creating a variable, you don't need to define which data type it is – you don't need to declare a variable
- Indentation in MicroPython is 2 spaces
- Python is an interpreted language and C++ is a compiled language

Conclusion

This was just a quick getting started guide to get you familiar with Python/MicroPython syntax. If you already know how to program in Python, programming in MicroPython is almost the same.

If you are used to program your electronics using Arduino IDE, you'll find that MicroPython is much simpler and less prone to syntax errors.

Finally, if this is your first-time programming digital electronics, we hope you enjoy MicroPython.

Interacting with GPIOs – Blinking an LED

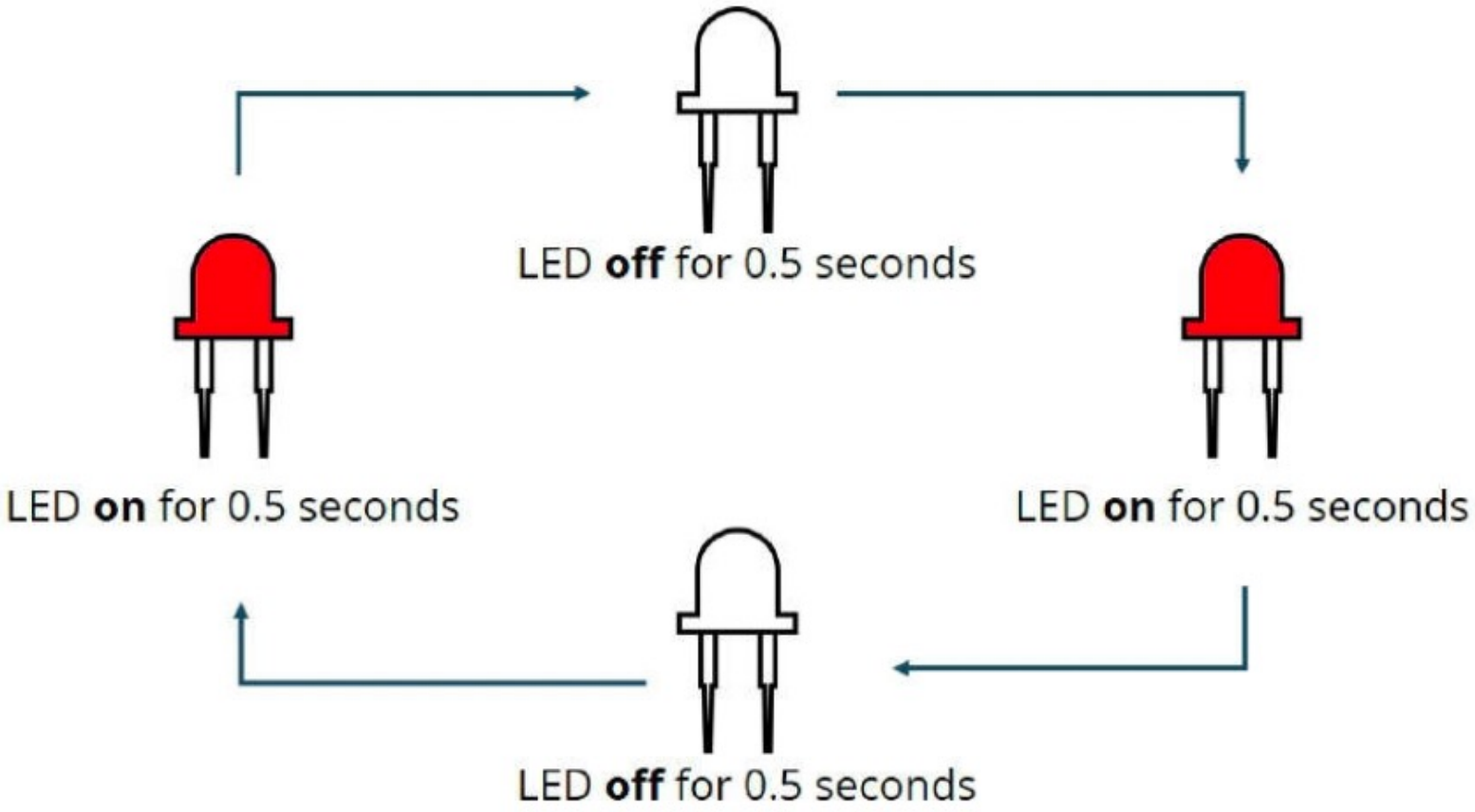
Project Overview

The project we'll build consists of simply blinking the ESP32/ESP8266 on-board LED. The on-board LED corresponds to GPIO 2 in both ESP32 and ESP8266. In simple terms, the blinking LED project works as follows:

- The LED turns on for 0.5 seconds—GPIO 2 set to HIGH.
- The LED turns off for 0.5 seconds—GPIO 2 set to LOW.
- The LED is on again for 0.5 seconds—GPIO 2 set to HIGH.
- The LED is off again for 0.5 seconds—GPIO 2 set to LOW.

This pattern continues until you tell the program to stop.

Interacting with GPIOs – Blinking an LED



Interacting with GPIOs – Blinking an LED

Script

The following script does precisely what we've described:

```
from machine import Pin
from time import sleep
led = Pin(2, Pin.OUT)
while True:
  led.value(not led.value())
  sleep(0.5)
```

Interacting with GPIOs – Blinking an LED

How the Code Works

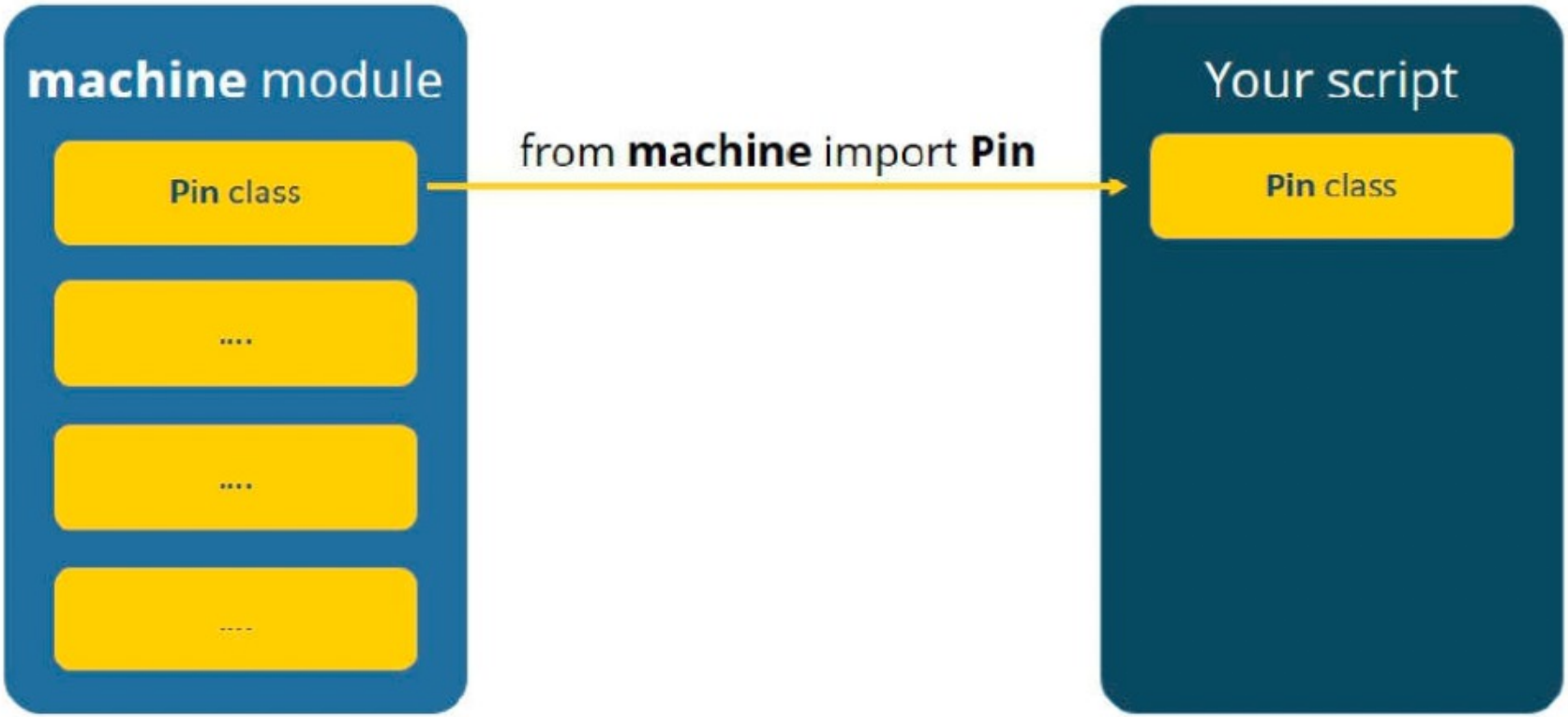
Let's take a closer look at the code to understand how it works.

The *machine* module

In all our scripts, we'll use the `machine` module. The `machine` module contains classes to interact with the physical world - this means classes to interact with the GPIOs.

The `machine` module contains many classes. We don't need all those classes in every script we create. In MicroPython, because we are writing code in constrained conditions, we should import only what we need. So, instead of importing all the classes in the `machine` module, we just need to import the `Pin` class in this example.

Interacting with GPIOs – Blinking an LED



The following line tells us to import only the `Pin` class from the `machine` module:

```
from machine import Pin
```

Interacting with GPIOs – Blinking an LED

The *time* module

In most of our scripts, we'll use the `time` module. The `time` module, as the name suggests, allows us to deal with time:

- get time and date;
- use delays;
- measure how many seconds have passed since the program started;
- etc.

In this script, we need to add a delay to create the blinking effect. To add a delay we need to use the `sleep()` method. Because we only need that method from the `time` module, we import it as follows:

```
from time import sleep
```

Interacting with GPIOs – Blinking an LED

Instantiating Pin objects

After importing all the necessary modules, we instantiate a `Pin` object called `led`.

The `Pin` object accepts these attributes in the following order:

```
(Pin number, pin mode, pull, value)
```

Here's what these arguments mean:

- **Pin number**: the GPIO number;
- **Pin mode**: a pin can be an input, output, or open-drain (we won't use open-drain mode in any of our examples):
  - **Pin.IN**
  - **Pin.OUT**
  - **Pin.OPEN_DRAIN**

Because we want GPIO 2 to act as an output, we need to set the mode to `Pin.OUT`.

Interacting with GPIOs – Blinking an LED

- **Pull**: this argument is used if we want to activate a pull up or pull-down internal resistor:
    - **Pin.PULL_UP** — pull-up
    - **Pin.PULL_DOWN** — pull-down

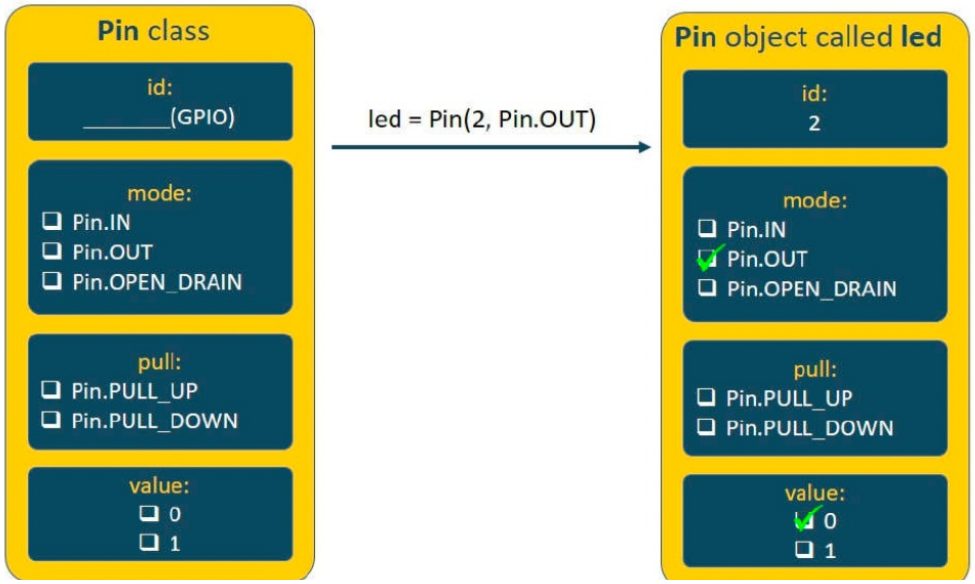
We won't set any internal resistor, so we don't need to pass this parameter.

- **Value**: can be 0 or 1, or True or False. Setting 0 or False means the GPIO is off. Setting 1 or True means the GPIO is on. If we don't pass any parameter, its state is 0 by default. So, our `led` object looks as follows:

```
led = Pin(2, Pin.OUT)
```

Interacting with GPIOs – Blinking an LED

The following figure may help you better understand how to create our `led` object from the `Pin` class.



Then, you create a `while` loop that is always `True`. This is a little trick to make something run forever and ever.

```
while True:
```

Interacting with GPIOs – Blinking an LED

Inside the loop, we set if the GPIO is on or off. The `Pin` class provides a method called `value()` that allows us to change the state of the GPIO. For example, to change the value of the led, you use the following:

```
led.value(state)
```

In which `state` corresponds to the state you want to set to the led: either 0 or 1 (or `True` or `False`).

Interacting with GPIOs – Blinking an LED

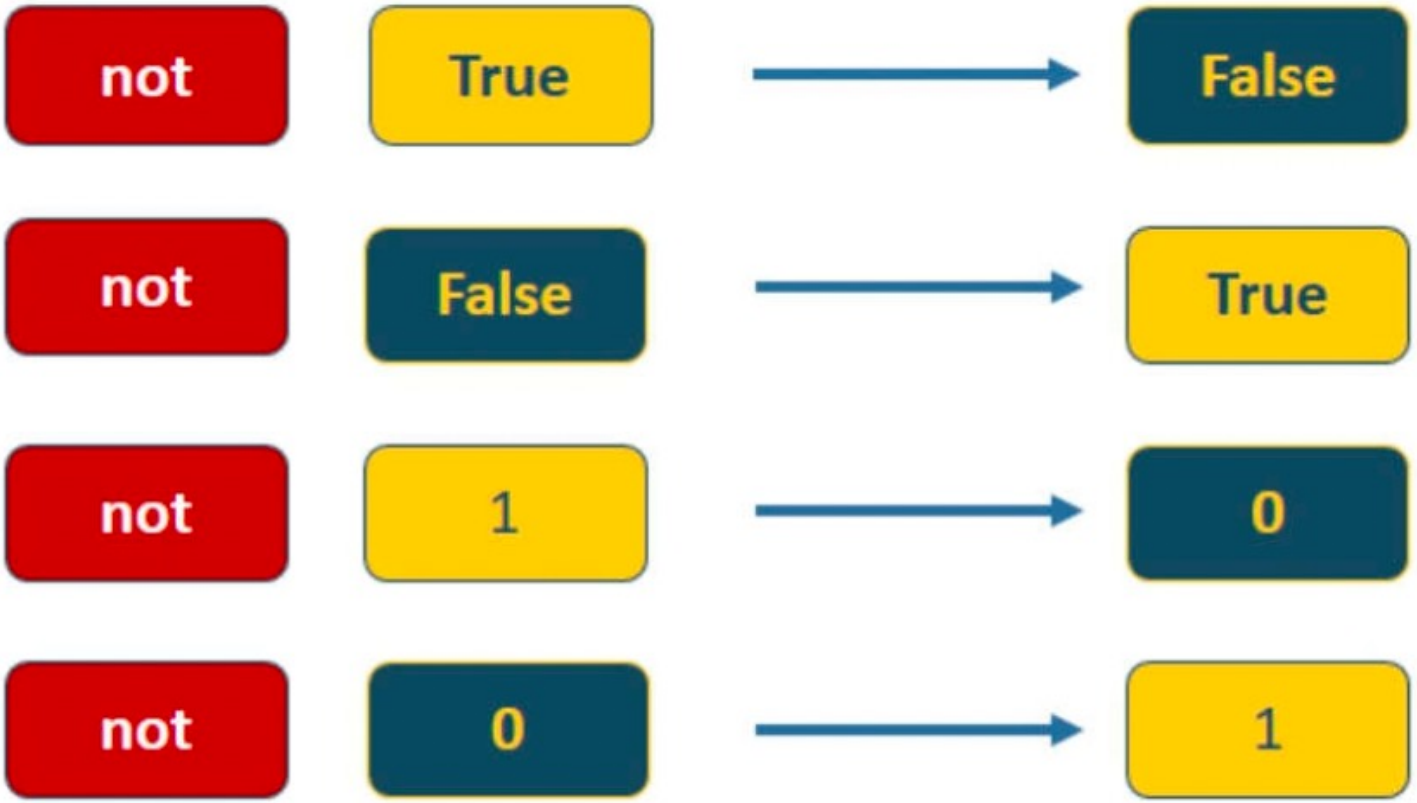
As argument we pass the following expression:

```
not led.value()
```

The `led.value()` expression (without any arguments) returns the current state of the `led` object. In the first loop `led.value()` is 0. By applying the ***not*** logical operator, we get 1.

Interacting with GPIOs – Blinking an LED

Not is a logical operator that reverses the state of the expression that follows. To better understand this concept, look at the following figure.



Interacting with GPIOs – Blinking an LED

After lighting up the LED, we wait for half a second (0.5 seconds).

```
sleep(0.5)
```

The `sleep()` function waits for the number of seconds you pass as argument. The code doesn't do anything else while you're in the `sleep()` function.

After waiting half a second, the loop restarts. This time, the `led.value()` is 1. By applying the `not` logical operator, it becomes 0. This is the same of writing `led.value(0)`, so the on-board LED turns off.

Then, we wait for half a second, and the loop repeats again and again, producing the blinking effect.

Interacting with GPIOs – Blinking an LED

The following table may help you better understand what is going on in the code.

	led.value()	not led.value()	led.value(not led.value())
Loop number 1	0	1	led.value(1) → GPIO ON
Loop number 2	1	0	led.value(0) → GPIO OFF
Loop number 3	0	1	led.value(1) → GPIO ON
...	...	...	...

Interacting with GPIOs – Blinking an LED

Upload the Code

Open uPyCraft IDE. Connect your ESP board to your computer using an USB cable. Make sure you select the right COM port in **Tools ▸ Serial**, and the right board in **Tools ▸ Board**. Then, upload the code to your board as we've shown in you in previous Units.

Demonstration

The on-board ESP32 LED should be blinking every half a second.

Thanks for your listening!

